IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received June 24, 2020, accepted July 24, 2020, date of publication July 28, 2020, date of current version August 10, 2020.

Digital Object Identifier 10.1109/ACCESS.2020.3012606

An Effective Constraint-Handling Improved Cuckoo Search Algorithm and Its Application in Aerodynamic Shape Optimization

JUN LIU, DOU WANG, AND SHIBIN LUO
School of Aeronautics and Astronautics, Central South University, Changsha 410083, China
Corresponding author: Dou Wang (wd8807@csu.edu.cn)

ABSTRACT This research develops an effective constraint-handling method to improve meta-heuristic algorithms' performance when solving constrained optimization problems. With the cuckoo search (CS) as the basic optimization algorithm, a constraint-handling improved cuckoo search algorithm (CICS) is presented and applied to an airfoil aerodynamic shape optimization. First, the newly developed constraint-handling (CH) method is compared to five types of traditional techniques by incorporating the cuckoo search algorithm, particle swarm optimization (PSO), and genetic algorithm (GA), respectively, on ten benchmark analytical test problems and four engineering design optimization problems. Results indicate that the present method is effective and robust, and outperforms the other constraint-handling methods, not only for cuckoo search but also for particle swarm optimization and genetic algorithm. Next, the presented constraint-handling improved cuckoo search algorithm is compared to nine types of state-of-the-art meta-heuristic algorithms. It shows better performance when solving the aforementioned constrained problems. Finally, the CICS algorithm is successfully applied to a strongly constrained aerodynamic drag minimization design problem. It also shows its feasibility to aerodynamic design optimization and superiority to the original cuckoo search algorithm.

INDEX TERMS Cuckoo search, engineering design optimization, constraint handling, aerodynamic shape optimization.

I. INTRODUCTION

Most engineering optimization design problems are often highly nonlinear, involving multiple complex constraints, which poses a challenge for optimization algorithms. Gradient-based algorithms tend to be efficient, but they require the derivative of the objective function to the design variables, which is often not easy to obtain. Besides, gradient-based algorithms are easy to trap to a local optimum, which leads to a reduction of design quality. As a result, a global optimization algorithm is usually more favored. For example, the swarm-based meta-heuristic algorithms, such as genetic algorithms (GAs) which is inspired from Darwin's natural selection theorem based on the idea of the survival of the fitness [1], [2]; particle swarm optimization (PSO) which is motivated from the social behavior of bird flocking and fish schooling [3]; ant colony algorithm (ACA) which mimics the foraging behavior of the ant colonies [4] and so on. Another type of global optimization algorithm is the surrogate-based optimization (SBO), in which the expensive cost function is replaced by a cheap approximation model during the optimization process (called sub-optimization) to improve the efficiency [5]–[7]. Since the effectiveness of sub-optimization is crucial to the performance of SBO [5], a higher performance global optimizer is also required.

With the rapid development of artificial intelligence technology, many excellent meta-heuristic optimization algorithms have been proposed in recent years. For example, artificial bee colony (ABC) algorithm [8], which is inspired by the foraging behavior of honey bees; flower pollination algorithm (FPA) which takes inspiration from the flower species found in nature [9]; tree seed algorithm (TSA), being inspired by the growing of the trees and seed production process [10]; henry gas solubility optimization (HGSO), which mimics the behavior governed by Henry's law to solve challenging problems [11]; the naked mole-rat algorithm (NMR) [12], which mimics the mating patterns of naked mole rats

The associate editor coordinating the review of this manuscript and approving it for publication was Manuel Rosa-Zurera.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

present in nature; slime mould algorithm (SMA) based upon the oscillation mode of slime mould in nature [13]; equilibrium optimizer (EO), inspired by control volume mass balance models used to estimate both dynamic and equilibrium states [14]; Harris hawks optimization (HHO) which is proposed to mimic the hunting behavior of Harris hawks [15], and so on. More algorithms have also been proposed, such as the moth-flame optimization algorithm (MFO) [16], the salp swarm Algorithm (SSA) [17], whale optimization algorithm [18], and so on.

Cuckoo search (CS) is one of the recent meta-heuristic global optimization algorithms [19]–[22]. It is inspired by the feeding behavior of some cuckoo species and Lévy flights. This algorithm is verified to be more efficient and effective than the current well-known global optimization algorithms, such as GA and PSO [20]. As a result, it gets widespread concern and rapid development since it is proposed. On the one hand, researchers are applying this new algorithm to a variety of fields, such as engineering design optimization [23]–[25], economic dispatch [26], [27], medical applications [28], energy [29], data mining and clustering [30], and image processing [31]; on the other hand, researchers are making their effort to improve the efficiency or effectiveness of the original CS algorithm, or broadening its scope of application. Hence, many variants of CS algorithms are developed: Gherboudj *et al.* [32] proposed a binary cuckoo algorithm (BCS) for binary optimization problems; Ouyang *et al.* [33] proposed a discrete cuckoo search algorithm (DCS) for solving spherical traveling salesman problem; Tuba *et al.* [34] proposed a modified cuckoo search (MCS) for unconstrained optimization problems, and Walton *et al.* [35] proposed another type of modified CS; Zhou *et al.* [36] proposed an improved cuckoo algorithm (ICS) to solve planar coloring problem; Kanagaraj *et al.* [37] proposed a hybrid cuckoo search and genetic algorithm (HCSGA) by embedding the genetic operators in the standard CS; Yang and Deb [38] extended the single-objective CS to multi-objective case and proposed a multi-objective cuckoo search algorithm (MOCS) to search the Pareto optimal front, they found that the MOCS is superior to NSGA-II, SPEA, and other multi-objective evolutionary algorithms. Zhang *et al.* [39] improved the original MOCS by implementing non-dominant sorting, crowding distance, and archive management strategies. For more improvements and a detailed review of the CS algorithms, the authors are referred to as references [40]–[43].

Real design optimization problems often involve complex constraints. Hence, some constraint-handling (CH) methods must be implemented in an optimizer to treat such problems. During the last decades, a lot of CH methodologies have been proposed for meta-heuristic algorithms. Coello [44] classified most CH techniques into five categories: (1) methods based on penalty functions; (2)methods based on preserving feasibility of solutions; (3) methods separating the objective and constraints; (4) methods based on decoders; and (5) hybrid methods. The penalty methods are the most commonly used CH techniques in which penalty functions are incorporated into the original objective function to penalize the constraint violations. In these methods, either a static penalty function [45], or dynamic function [46], or adaptive function [47] is used. Deb [48] proposed a pairwise comparison without any penalty parameter in tournament selection for a genetic algorithm. In his method, when comparing pairwise individuals, (1) a feasible solution always outperforms an infeasible one; (2) if the two solutions are both feasible, the one with better objective function wins; (3) if both are infeasible, the one with less constraint violation is chosen. Inspired by this method, Wang *et al.* [47] treated the population as three situations: an infeasible situation if all solutions are infeasible in the population, a semi-feasible situation when both feasible and infeasible solutions are contained in the population, feasible situation if the population is composed of only feasible solutions. A different mechanism of constraint-handling is used for a different situation. Runarsson and Yao [49] proposed a stochastic ranking (SR) method to balance the objective and penalty functions. In this method, a parameter p_f was introduced to switch between the objective value and the constraint violations when comparing two solutions in the sorting process. A satisfaction level for the constraints was introduced by Takahama and Sakai [50] to indicate how well a solution satisfies the constraints. The satisfaction level and objective function were used to select the solutions. This level comparison algorithm was incorporated into the differential evolution algorithm (DE) for optimization problems with constraints by Wang and Li [51]. Another type of CH technique is to treat the constraints as one or more objectives, and multi-objective optimization algorithms subsequently solve the original single objective problem. Cai and Wang [52] proposed one more constraint-handling method by drawing the concept of Pareto dominance, and very competitive results were produced. Venter and Haftka [53] treated the sum of all the constraints as an additional objective function and transformed the constrained single-objective optimization problem into a bi-objective unconstrained optimization problem.

The original CS algorithm uses the static penalty method to treat constrained problems. However, there are at least two drawbacks to the penalty methods. First, they have user-defined parameters, that is the penalty factor. The selection of the penalty factors will significantly affect the quality of optimization, and the best penalty factors often vary considerably due to different optimization problems. Hence, it is not easy to obtain an ideal result for an unknown, constrained real engineering problem. Second, there are usually multiple constraint functions for an engineering design problem. The scale of each constraint function may be quite different, which will increase the difficulty of determining the penalty factors for different constraint functions. Although there have been proposed a lot of modified or improved CS algorithms in recent years, almost all of them are aimed at enhancing the optimization quality for unconstrained problems. At the same time, few of them are primarily focused on the improvements for constrained problems. As a result, more work should be done to improve the optimization quality for constrained problems.

Motivated by these, this paper proposes an enhanced CH method and a constrain-handling improved cuckoo search algorithm (CICS) to improve the optimization quality for constrained problems.

Two strategies are introduced to strengthen the constraint handling of the original CS algorithm. First of all, the separation method (in which the objective function and constraint functions are treated separately) is introduced to eliminate the penalty factors. Secondly, to reduce the influence of the order of magnitude of different constraint functions, each constraint function in the population is regularized and then summed to ensure that all constraint functions of each individual are in the same order of magnitude at each iteration. Thirdly, when comparing a new solution with an old one, a dominance mechanism is introduced to ensure that the individuals with less constraint violation will inherit to the next iteration. The above CH method is incorporated into the cuckoo search; hence the CICS is presented. The new CH method is verified by comparing to five types of existing traditional CH methods by integrating to the CS, particle swarm optimization (PSO), and genetic algorithm (GA), respectively. Then, the presented CICS algorithm is compared to several state-of-the-art meta-heuristic optimizations, and it shows excellent performance when dealing with constrained problems. Finally, the presented CICS algorithm is further applied to a strongly constrained aerodynamic shape optimization problem, which explains its feasibility in the aerodynamic design field and superiority to the original cuckoo search.

The article is divided into seven sections. It is outlined as follows: Section I details the introduction, giving the background of this research, motivation, objectives, and contribution of this research. Section II provides a short introduction to the original cuckoo search. In Section III, the presented constrained handling method and improved cuckoo search algorithm are described. Section IV validates the presented methods by benchmark analytical problems and engineering problems. Then, the presented CICS algorithm is further confirmed by an aerodynamic shape optimization problem in Section V. The main findings of the research, limitations, and insightful amplifications from the proposed work are presented in Section VI. The last section concludes the whole research work.

II. BRIEF INTRODUCTION OF CUCKOO SEARCH ALGORITHM

The cuckoo search algorithm is a new and efficient nature-inspired meta-heuristic optimization algorithm developed by Yang and Deb [19] in 2009. It is based on the obligate brood parasitic behavior of some cuckoo species and the Lévy flight behavior of some birds and fruit flies. Besides having higher efficiency and effectiveness, the CS has two more advantages over traditional population-based optimization algorithms, which is easy for implementation and has few user-defined parameters [54]. The three main principal rules during the search process of this algorithm are shown as follows [19].

(1) Each cuckoo lays one egg at a time and dumps it in a randomly chosen nest.

(2) The best nests with high-quality eggs (solutions) will be carried over to the next generations.

(3) The number of available host nests is fixed, and a host bird can discover an alien egg with a probability $p_a \in [0, 1]$. In this case, the host bird can either throw the egg away or abandon the nest so as to build a completely new nest in a new location.

The last assumption can be further approximated by a fraction p_a of the n host nests that are substituted by newly built nests (newly generated random solutions). For maximizing problems, the solution's quality or fitness can be proportional to the objective function value. For minimizing issues, it can be equivalent to the negative amount of the objective function. At the same time, other forms of fitness function can also be used. Only minimization problems are considered in this research since maximization can be transformed into minimization.

Most engineering design optimization problems consist of complex nonlinear constraints. For a common nonlinear optimization problem with equality and inequality constraints, the formulation can be expressed as follows.

$$\begin{aligned} &Minimize: f(\boldsymbol{x}), \boldsymbol{x} = (x_1, \cdots, x_d)^T \in R^d \\ &Subject\ to: \ g_j(\boldsymbol{x}) \le 0, (i = 1, \cdots, p) \\ &\qquad h_i(\boldsymbol{x}) = 0, \quad (j = p+1, \cdots, m). \end{aligned} \tag{1}$$

In the original CS, a static penalty is introduced to handle the constraints [22]. The fitness function is defined as

$$\text{fitness}: F(\boldsymbol{x}) = f(\boldsymbol{x}) + \sum_{i=1}^{p} \mu_i [\max(0, g_i(\boldsymbol{x}))]^2 + \sum_{j=p+1}^{m} v_j h_j^2(\boldsymbol{x}) \tag{2}$$

where $\mu_i \ge 0$ and $v_i \gg 1$ which should be large enough, depending on the solution quality needed. Yang [22] suggests that for most applications, μ_i and v_j can be set to 10^{10} to 10^{15}. Actually, this penalty method belongs to the static penalty which will be shown in part IV.

FIGURE 1 shows the pseudo code of the original CS for solving a common optimization problem.

III. CONSTRAINED HANDLING IMPROVED CUCKOO SEARCH ALGORITHM

Most recent meta-heuristic algorithms haven't considered the performance for the application of constrained optimization problems. Even used for constrained problems, the traditional constraint handling methods are used, in which the penalty method is the favorite. As mentioned before, the original CS algorithm doesn't contain a specific constraint handling method, and the proposer of the algorithm introduces the static penalty. Hence, a penalty factor should be set to treat a constrained optimization problem in the original cuckoo search. Although Yang [22] suggests that for most issues,

01 Objective function $f(\boldsymbol{x}), \boldsymbol{x}=(x_1,\cdots,x_d)^T$;
02 Generate initial population of n host nests $\boldsymbol{x}_i$;
03 While $(t < MaxGeneration)$ or (other stop criterion)
04 Get a cuckoo randomly/generate a solution by Lêvy flights
05 and then evaluate its quality/fitness F_i;
06 Choose a nest among n(say, j) randomly;
07 if $(F_i < F_j)$ then
08 Replace j by the new solution;
09 end
10 A fraction (p_a) of worse nests are abandoned
11 and new ones/solutions are built/gengrated;
12 Keep best solutions(or nests with quality solutions); (by fitness function)
13 Rank the solutions and find the current best; (by fitness function)
14 endwhile
15 Postprocess results and visualization;

FIGURE 1. Pseudo code of the original cuckoo search algorithm [20].

the penalty factor can be set as 10^{10} to 10^{15}. However, according to the authors' practice, with a specified maximum number of function evaluation, the penalty factor will significantly affect the optimization results: if the penalty factor is too small, the optimizer may find an infeasible solution, while if it is too large, the obtained optimal solution may not be satisfactory. It is usually not easy to find the best value of it. Hence, a new CH method is introduced in this research to reduce the selection of penalty factors on the optimization result.

To solve the constrained problem in Eq.(1), the equality constraints can be transformed into inequality form, and the constraint violation is defined as [55]:

$$G_i(\boldsymbol{x}) = \begin{cases} \max\{g_i(\boldsymbol{x}), 0\}, & i = 1, \cdots, p \\ \max\{|h_i(\boldsymbol{x})| - \varepsilon, 0\}, & i = p+1, \cdots, m \end{cases} \quad (3)$$

where ε is a tolerance of the equality constraints (a small positive value). Therefore, the target is to minimize the objective function $f(\boldsymbol{x})$ subject to all the inequality constraints. The total constraint violation for an infeasible individual is a weighted average of all the constraints, which is expressed as

$$v(\boldsymbol{x}) = \frac{\sum_{i=1}^{m}(\omega_i \cdot G_i(\boldsymbol{x}))}{\sum_{i=1}^{m}\omega_i}, \quad (4)$$

in which $\omega_i = 1/G_{\max,i}$ is a weight parameter, $G_{\max,i}$ is the maximum violation of ith constraint obtained so far. Here ω_i is set to $1/G_{\max,i}$ which varies during the iteration process in order to balance the contribution of each constraint irrespective of their differing numerical ranges. After the objective function value and total constraint violation are calculated, the superiority of the two solutions is determined by the dominant relationship as

$$\boldsymbol{x}_j \succ \boldsymbol{x}_i \begin{cases} if\ \left(0 \le v_j(\boldsymbol{x}) < v_i(\boldsymbol{x})\right) \text{or} \\ if\ \left(f(\boldsymbol{x}_j) < f(\boldsymbol{x}_i) \text{and} v_i = 0, v_j = 0\right) \end{cases} \quad \text{or} \quad (5)$$

where $\boldsymbol{x}_j \succ \boldsymbol{x}_i$ denotes that $\boldsymbol{x}_j$ dominates $\boldsymbol{x}_i$ or $\boldsymbol{x}_j$ is superior to $\boldsymbol{x}_i$. The Eq.(5) indicates that two cases may appear when comparing the superiority of two solutions: first, if at least one of the two solutions is infeasible, the one with less constraint violation is considered to be better; second, if both are feasible, the one with smaller objective function value is preferred.

Compared to Deb's [48] method, the above process only needs the two competing individuals' objective function value and total constraint violation while the worst objective function value f_{worst} of the population is not required; besides, the fitness function is thoroughly eliminated in this method. Two more difference between the present and Deb's algorithm are: first, different constraints are scaled to the same order of magnitude in this algorithm; second, this algorithm additionally introduces a ranking rule to rank all of the solutions in the population, as will be shown in the following, while Deb's method is only used in a pairwise comparison.

The presented constraint-handling process replaces the penalty method in the original cuckoo search algorithm. First, the row 07-09 of the cuckoo search in FIGURE 1 is replaced by Eq. (5) to determine the superiority of two solutions. Then, when ranking the nests (solutions) in the population in row 10-13, the constraints are treated by the following ranking rule:

(1) If all of the solutions in the population are infeasible, the solutions are ranked only by the value of total constraint violation.

(2) If all of the solutions are feasible, the solutions are ranked only by the objective function value;

(3) If both feasible and infeasible solutions exist, the feasible ones and infeasible ones are ranked separately.

As a result, a constraint-handling improved cuckoo search algorithm (CICS) is presented, which is summarized in FIGURE 2.

01 Objective function $f(\boldsymbol{x}), \boldsymbol{x}=(x_1,\cdots,x_D)^T$;
02 Generate initial population of n host nests $\boldsymbol{x}_i$;
03 While $(t < MaxGeneration)$ or (other stop criterion)
04 Get a cuckoo randomly/generate a solution by Lêvy flights
05 and then evaluate its objective f_i and unified constraint v_i (*Eq.*4);
06 Choose a nest among n(say, i) randomly;
07 if $(\boldsymbol{x}_j \succ \boldsymbol{x}_i)$ then
08 replace solution i by the new solution j; (*Eq.*5)
09 end
10 A fraction (p_a) of worse nests are abandoned
11 and new ones/solutions are built/gengrated; (ranking rule)
12 Keep best solutions(or nests with quality solutions); (ranking rule)
13 Rank the solutions and find the current best; (ranking rule)
14 endwhile
15 Postprocess results and visualization;

FIGURE 2. Pseudo code of constraint handling improved Cuckoo search.

Comparing FIGURE 1 and FIGURE 2, one can see that there are mainly four differences between CICS and the original CS. First, in the original CS, a unified fitness function

is calculated, while in the new algorithm, the objective value and constraint functions are calculated separately. Second, the sum of constraints with the user-defined parameter (Eq.(2)) is replaced by a total constraint violation without any user-defined parameter (Eq.(4)). Third, in the original CS, the unified fitness function is directly compared, while in CICS, two solutions are compared using a dominance mechanism defined in Eq.(5). Last, the fitness function is used to rank the solutions in the population for CS, while for the CICS, a ranking rule based on the dominance mechanism is used.

The computational complexity of the original CS can be given by $O(n.D.t_{\max})$, where n is the population size, D is the dimension size, and $t_{\max}$ is the maximum number of iterations [40]. Considering the first and second differences, as shown above, when Eq.(2) is replaced by Eq.(4), the computational burden is the same. While for the third and fourth differences, the fitness comparison is replaced by Eq.(5). There is one computation for the former, while two calculations for the latter, but the complexity is both $O(1)$. Based on the above analysis, the improved algorithm's complexity is almost the same as that of the original CS algorithm.

IV. VALIDATION

A. OPTIMIZATION CASES

Ten benchmark test problems and four engineering optimization problems are employed to validate the proposed constraint-handling method and the constraint-handling improved cuckoo search algorithm. For each problem, the target is to minimize an objective function subject to several constraint functions.

1) ANALYTICAL TEST CASES

The ten well-known benchmark problems are introduced from Runarsson and Yao [49]. The main features of the problems are shown in TABLE 1, including dimensionality, type of objective function, number of constraints, number of active restraints, and feasibility percentage. The feasibility percentage denotes the portion of feasible regions in the total decision space, which is approximated using the computer experiment: one million solutions are randomly generated, and the percentage of viable solutions is calculated [37]. The mathematical formulation of the problems can be seen in the Refs [49].

2) ENGINEERING OPTIMIZATION PROBLEMS

The performance of the proposed constraint-handling method and the CICS algorithm is further validated through four benchmark structural design optimization problems. They are the welded beam design problem (W.B.), tension/compression spring design (S.D.), speed reducer design problem (S.R.), and pressure vessel design problem (P.V.). These problems are real-world optimization design problems. They all have explicit formulation and known theoretical optimal solutions, hence widely used to test the performance of optimization algorithms [3], [37].

B. ALGORITHMS FOR COMPARISON

During the last decades, a variety of CH methods have been proposed for evolutionary optimization algorithms, which have been well-reviewed by Coello [44]. In this paper, five types of existing CH methods are introduced to compare with the present CH process, by applying to CS, GA, PSO, respectively. They are the four types of penalty methods and Deb's approach [48].

TABLE 1. Main features of the 10 benchmark problems.

Problem	dimensionality	Type of objective function	Feasibility percentage	No. of constraints	No. of active constraints
G01	13	Quadratic	0.0003	9	6
G02	20	Nonlinear	99.9975	2	1
G03	10	Nonlinear	0.0000	1	1
G04	5	Quadratic	26.9356	6	2
G05	4	Nonlinear	0.0000	5	3
G06	2	Nonlinear	0.0064	2	2
G07	10	Quadratic	0.0003	8	6
G08	2	Nonlinear	0.8640	2	0
G09	7	Nonlinear	0.5256	4	2
G10	8	Linear	0.0005	6	3

1) STATIC PENALTY

In this method, the constraint violation is multiplied by penalty factors and added to the original objective value to measure the performance of an individual, and the penalty factors remain constant during the entire optimization process. The formulation of fitness is

$$fitness\,(\boldsymbol{x}) = f\,(\boldsymbol{x}) + \sum_{i=1}^{m} \left(R_i \times \max\left[0, g_i\,(\boldsymbol{x})\right]^2\right), \quad (6)$$

where R_i are the penalty factors, m is the total number of constraints.

2) DYNAMIC PENALTY

In this method, the current generation (or iteration) number is involved in the computation of the corresponding penalty factors. A typical formulation of the fitness of the dynamic penalty is

$$fitness\,(\boldsymbol{x}) = f\,(\boldsymbol{x}) + (C \times t)^{\alpha} \times SVC\,(\beta, \boldsymbol{x}) \quad (7)$$

where α, β and C are user-defined constants and SVC is defined as

$$SVC\,(\beta, \boldsymbol{x}) = \sum_{i=1}^{p} D_i^{\beta}\,(\boldsymbol{x}) + \sum_{j=p+1}^{m} D_j\,(\boldsymbol{x}) \quad (8)$$

and

$$D_i(\boldsymbol{x}) = \begin{cases} 0, & g_i(\boldsymbol{x}) \le 0 \\ |g_i(\boldsymbol{x})|, & \text{otherwise}, \end{cases} \quad 1 \le i \le p, \tag{9}$$

$$D_j(\boldsymbol{x}) = \begin{cases} 0, & |h_j(\boldsymbol{x})| \le \varepsilon \\ |h_j(\boldsymbol{x})|, & \text{otherwise}, \end{cases} \quad p+1 \le j \le m. \tag{10}$$

where ε is the tolerance of equality constraints.

3) ADAPTIVE PENALTY

This method updates the penalty function according to the feedback of the search process. The fitness of each individual is evaluated by the following formulation:

$$fitness(\boldsymbol{x}) = f(\boldsymbol{x}) + \lambda(t)\left[\sum_{i=1}^{p} g_i^2(\boldsymbol{x}) + \sum_{j=p+1}^{m} |h_j(\boldsymbol{x})|\right], \tag{11}$$

where $\lambda(t)$ is updated at every iteration t in the following way:

$$\lambda(t+1) = \begin{cases} (1/\beta_1)\cdot\lambda(t), & \text{if case \# 1} \\ \beta_2\cdot\lambda(t), & \text{if case \# 2} \\ \lambda(t), & \text{otherwise} \end{cases} \tag{12}$$

where case #1 and case #2 denotes situations where the best individuals in the last k generations were always (case #1) or were never (case #2) feasible, $\beta_1 > \beta_2 > 1$. That is to say, the penalty factor $\lambda(t+1)$ for the t+1 iteration will be decreased if all the best solutions in the last k iterations were feasible or it will be increased if they were all infeasible. If there are both feasible and infeasible solutions kept unchanged as best in the population, then the penalty factor will not be changed.

4) ANNEALING PENALTY

This method is firstly proposed for the simulated annealing algorithm: the penalty coefficient is changed once in many iterations. At each iteration of the searching process, only the active constraints are considered, and the penalty is increased during the iteration process. The individuals are evaluated using

$$fitness(\boldsymbol{x}) = f(\boldsymbol{x}) + \frac{1}{2\tau}\sum_{i=1}^{m}\phi_i^2(\boldsymbol{x}) \tag{13}$$

where τ is the cooling schedule, and

$$\phi_i(\boldsymbol{x}) = \begin{cases} \max[0, g_i(\boldsymbol{x})], & \text{if } 1 \le i \le p \\ |h_j(\boldsymbol{x})|, & \text{if } p+1 \le j \le m \end{cases} \tag{14}$$

5) DEB'S METHOD

Deb [48] proposed an approach where the fitness of an individual is:

$$fitness(\boldsymbol{x}) = \begin{cases} f(\boldsymbol{x}), & \text{if } g_i(\boldsymbol{x}) \le 0, \forall i = 1, \cdots, m \\ f_{worst} + \sum_{i=1}^{m}\langle g_i(\boldsymbol{x})\rangle, & \text{otherwise} \end{cases} \tag{15}$$

where f_{worst} is the largest objective function value among all feasible solutions in the population and $\langle g_i(\boldsymbol{x})\rangle$ returns $g_i(\boldsymbol{x})$ if it is positive while returns zero otherwise. Besides, $g_i(\boldsymbol{x})$ refers only to inequality constraints (the inequality constraints are transformed to inequality constraints); if there is no feasible solution in the population, then f_{worst} is set to zero. Deb used this formulation in the tournament selection of the genetic algorithm. For this method, no penalty factor is required, since only the pairwise comparison is performed in the selection procedure.

After incorporating all of the above standard constraint-handling methods to the CS to compare with the newly proposed CH method, we then combine them and the proposed approach to the most commonly used meta-heuristic algorithms: GA and PSO. For each optimization algorithm, the proposed constraint-handling method is compared with the five methods mentioned above. In this research, a real coded GA is used, in which the tournament selection, arithmetic crossover, and non-uniform mutation are selected. For the PSO, the standard version is employed.

Each test problem is run with six types of different constraint-handling methods and different optimization algorithms, respectively. The algorithms for comparison are listed in TABLE 2.

TABLE 2. Algorithms for comparison.

algorithm	Static penalty	Dynamic penalty	Adaptive penalty	Annealing penalty	Deb's method	Present method
CS	CS-static	CS-dynamic	CS-adaptive	CS-annealing	CS-Deb	CS-present (CICS)
PSO	PSO-static	PSO-dynamic	PSO-adaptive	PSO-annealing	PSO-Deb	PSO-present
GA	GA-static	GA-dynamic	GA-adaptive	GA-annealing	GA-Deb	GA-present

After comparing the constraint-handling methods, the proposed constraint-handling improved cuckoo search (CICS) is subsequently compared with nine other meta-heuristic algorithms, including the enhanced GA, strengthened PSO, modified PSO, weighted differential evolution, and some different recent meta-heuristic optimization algorithms.

C. PARAMETER SETTING AND PERFORMANCE MEASURES

1) PARAMETER SETTING

There are two user-defined parameters for the cuckoo search algorithm: the nest number n and the probability of alien egg

discovery p_a. It is found that $n = 15$ to 25 and $p_a = 0.15$ to 0.30 are sufficient for most optimization problems and $n = 25$ and $p_a = 0.25$ are suggested in the research of [19], [20], [22]. As a result, the nest number and probability of alien egg discovery are set to 25 and 0.25, respectively, in this study.

For the PSO, the user-defined parameters are set according to Shi [56]: the acceleration constants c_1 and c_2 both equal to 2.0, the maximum velocity $Vmax$ is set to 20% of the dynamic range of the variable on each dimension, and the population size is set to 30 during the whole procedure in this study.

For the GA, the parameters are set according to some experiences: the population size n is set to 100, the probability of crossover p_c and mutation p_m is set to 0.8 and 0.1, respectively. Moreover, we find that the optimization results are not very sensitive to the setting of these parameters.

It is noted that the constraint-handling method proposed in this article and Deb's technique have no additional parameters. However, there are additional user-defined parameters for other technologies, and it is found that they affect the optimization results dramatically. As a result, it is challenging to find the best choice of these parameters since they vary among optimization problems and algorithms. Here, we follow Coello [44] to determine these parameters and keep them fixed for different cases. Although these values are not necessarily optimal for a specified problem, it is reasonable since we rarely know the best parameters beforehand for an unknown issue in real applications. Consequently, by following Coello [44], the setting for dynamic penalty is $C = 1.0$, $\alpha = \beta = 2.0$, parameters for the adaptive penalty is $\beta_1 = 2.5$, $\beta_2 = 2$, $k = 20$, $\lambda_0 = 1000$, parameters for annealing penalty are $\tau_f = 0.000001$, $\tau_0 = 1.0$, and τ is updated every 20 generations, the penalty factor (PF) for the static penalty is 10^N (N is an integer) and $N = 10$. Moreover, the best N for each problem will be found by the exhaustion method, and the corresponding optimization results will also be given.

2) PERFORMANCE MEASURES

Due to the randomness of meta-heuristic algorithms (random number generators used in these algorithms), each optimization case is repeated 100 times independently. The statistical value, i.e., means ($\bar{f} - f^*_{\min}$, where $f^*_{\min}$ denotes the real global optimum), standard deviations (σ), probability of satisfying the constraints (feasibility ratio), and the CPU time (Intel i7-8700 CPU, 3.2GHz) are obtained. Note that the feasibility ratio (FR) refers to the proportion of feasible solutions out of the 100 runs rather than the feasibility percentage of the problem itself, which is shown in TABLE 1.

In addition to the above traditional statistical approach, a more robust method that is the hypothetical test, is employed to compare the algorithms. In this work, the Wilcoxon signed-rank test [57] is used to compare the proposed method to the existing methods. This is a non-parametric process for hypothesis testing and involves design with two samples. It is similar to the paired t-test in non-parametric statistical procedures; hence, it is a pairwise test designed to detect the significant difference between the mean value of two samples, that is, the superiority of the two algorithms. Let us define the following hypotheses:

$$\begin{aligned} H_0 &: M_+ = M_- \\ H_1 &: M_+ > M_- \end{aligned} \tag{16}$$

where H_0 is the null hypothesis, and H_1 is the alternative hypothesis. In this paper, M_+ denotes the negative value of the optimum obtained by the present constraint-handling method, while M_- denotes the negative value of optimum obtained by another method. The reason for using negative value is that for a minimization problem, the smaller, the better of the objective value, hence the larger, the better for the negative objective value. Besides, when comparing solutions obtained by different algorithms, a feasible solution always outperforms an infeasible one. Hence the following formulation for M is used:

$$\begin{aligned} M &= -\left[\left(f - f^*_{\min}\right) + 10^{10} \times (1 - flag)\right] \\ flag &= \begin{cases} 1, & \text{for a feasible solution} \\ 0, & \text{for an infeasible solution} \end{cases} \end{aligned} \tag{17}$$

where f is the objective function value obtained by the optimizer and $f^*_{\min}$ denotes the real global optimum.

Wilcoxon's test is executed as follows. Let d_i be the difference between the performance scores of the two algorithms on ith out of n runs. The differences are ranked according to their absolute values; in case of ties, we use average ranks (for example, if two differences are overlapped in the assignation of ranks 1 and 2, they are both assigned to rank 1.5). We define R^+ be the sum of ranks for the problems where the first algorithm outperformed the second, and R^- the sum of ranks for the opposite. Besides, ranks of $d_i = 0$ are split evenly among the sums; if there is an odd number of them, one is ignored. The expression of R^+ and R^- are:

$$\begin{aligned} R^+ &= \sum_{di>0} rank\,(d_i) + \frac{1}{2}\sum_{di=0} rank\,(d_i) \\ R^- &= \sum_{di<0} rank\,(d_i) + \frac{1}{2}\sum_{di=0} rank\,(d_i). \end{aligned} \tag{18}$$

Let T be the smaller of R^+ and R^-, $T = \min\left(R^+, R^-\right)$. If T is less than or equal to the value of the Wilcoxon distribution for n degrees of freedom, the null hypothesis of equality of means is rejected; this will mean that a given algorithm outperforms the other one, with the p-value associated. The p-value is computed by the statistic and machine learning toolbox of MATLAB in this study. The level of significance $\alpha = 0.1$, if the p-value is smaller than 0.1, the null hypothesis (H_0) is rejected, and the alternative hypothesis (H_1) is approved. That is to say, the present method significantly outperforms the other one.

D. RESULTS

The proposed constraint-handling method is compared with five typical existing methods by applying first to the cuckoo search, followed by the PSO and GA. For all test problems,

the maximum number of objective function evaluations is set to 15000, since the number of function evaluations is usually limited to several thousand or less in most real engineering design problems [35], especially in aerodynamic shape design optimization.

1) APPLICATION OF CUCKOO SEARCH ALGORITHM

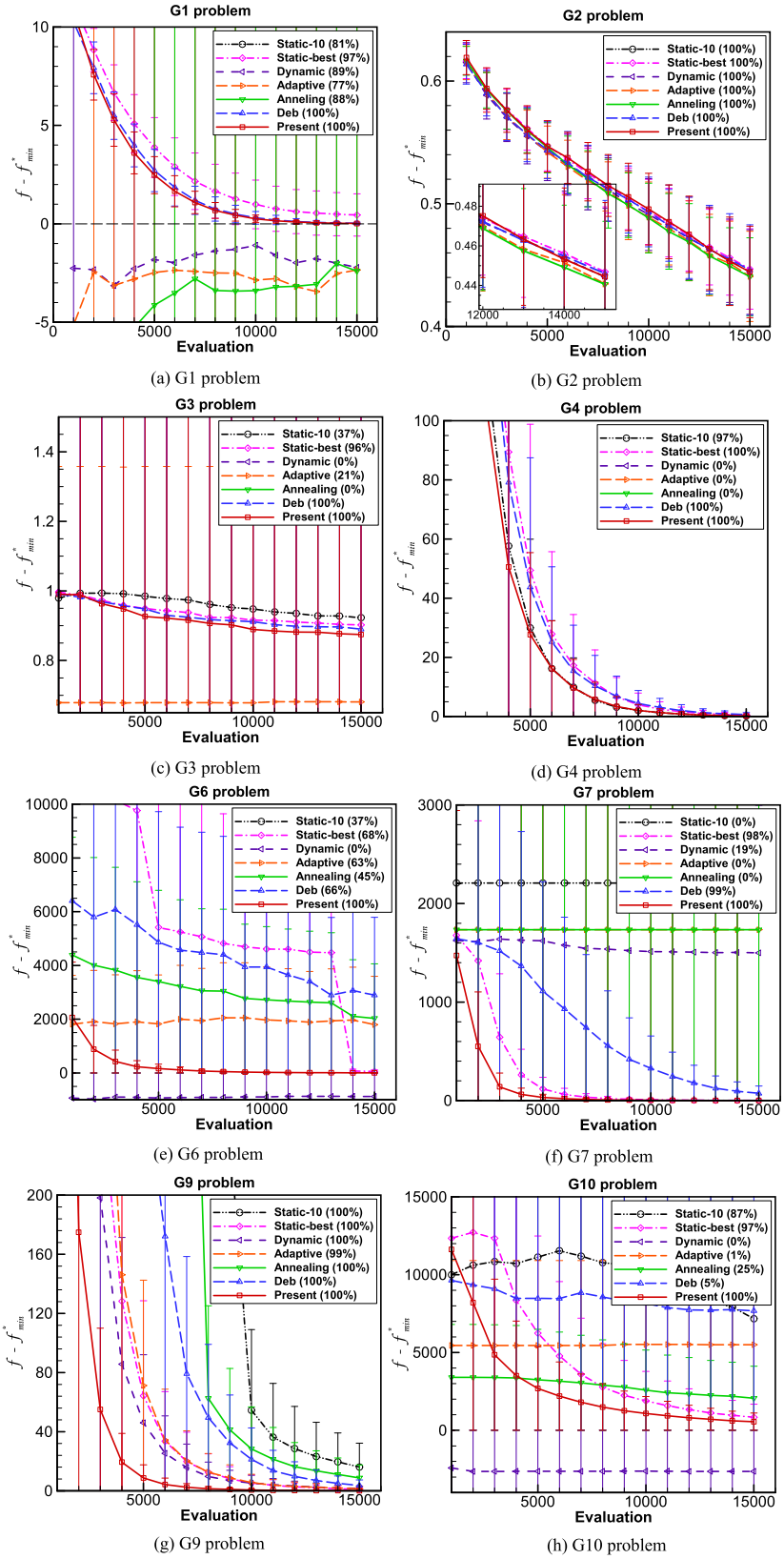
Before comparing the proposed CH method to the traditional CH methods, each modification of the present CH method is studied. Section III shows that there are mainly two modifications of the proposed CH method when comparing to the technique in the original CS: the scaling of different constraint functions shown in Eq.(4) and the dominance mechanism, which is shown in Eq.(5). TABLE 3 shows the results obtained by the original CH method, the CH method with only the first modification (scaling of each constraint), the CH method with only the second modification (dominance mechanism), and the combining of the two (the final version the present CH method). In this table, the averaged objective function ($\bar{f} - f^*_{\min}$) of the 100 runs, standard deviation, feasibility ratio, and the CPU time is shown. One can see that the feasibility ratio is significantly improved for each of the modifications and the combination of the two (G01, G04, G06, G7, G10, and Speed reducer problem). Besides, for most problems, each of the two modifications gets the benefit compared to the original CH method but performs not as good as the combination of the two. The results verify that each of the two changes of the proposed CH method is successful, and the combination of them also improves the final performance.

Then, the proposed CH method is compared to five types of existing five methods. FIGURE 3 shows the convergence histories of the optimization process on benchmark problems, where the presented constraint-handling method, as well as the five existing methods, are compared. The lines represent the averaged objective function ($\bar{f} - f^*_{\min}$) of the 100 runs. The vertical bars correspond to the standard deviation of the objective function (σ), the length of a vertical bar equals σ with its center located at $\bar{f} - f^*_{\min}$. The percentage score in the figures represents the percentage of satisfying the constraints for the 100 independently repeated optimization results. Note that the ideal value of $\bar{f} - f^*_{\min}$ is 0, which means the accurate global optimum is always found for each independent run; if it is smaller than 0, some of the optimal solutions out of the 100 runs are infeasible. The lines with the legend "Static-best" denote the results with the best penalty factors.

From these figures, one can see that the proposed method converges faster than all the other techniques for all the cases except the G2 problem. Besides, only the proposed method can always get feasible solutions for all the instances (the feasibility ratio is always 100%). Note that some methods get negative objective value (e.g., G1 problem); the reason is that they get infeasible solutions out of the 100 runs, which are undesired. For the G2 problem, the present method fails

TABLE 3. Comparison of each modification of the present CH method.

problem	type	Static (PF=10^{10})	Scaling (PF=10^{10})	Dominance	Present
G01	mean	-1.6111E+01	-8.2673E+00	4.9764E-02	1.7600E-02
	STD	4.8559E+01	3.9616E+01	2.6467E-01	1.6002E-02
	FR	81%	91%	100%	100%
	Time	1.744E-01	1.8954E-01	1.847E-01	1.794E-01
G02	mean	4.5188E-01	4.2119E-01	4.1822E-01	4.4445E-01
	STD	3.4273E-02	3.6925E-02	3.1206E-02	3.7344E-02
	FR	100%	100%	100%	100%
	Time	1.973E-01	2.1688E-01	2.127E-01	2.020E-01
G03	mean	9.2309E-01	9.3992E-01	8.8676E-01	8.7471E-01
	STD	9.1959E-02	8.3727E-02	1.1670E-01	1.2623E-01
	FR	100%	100%	100%	100%
	Time	1.491E-1	1.4549E-01	1.418E-01	1.493E-1
G04	mean	2.0900E-01	2.0087E-01	2.0077E-01	1.8857E-01
	STD	2.0678E-01	2.4189E-01	1.8531 E-01	1.8927E-01
	FR	97%	100%	100%	100%
	Time	1.530E-01	1.6805E-01	1.6670E-01	1.675E-01
G06	mean	1.1293E+05	1.4294E+01	2.8341E+00	2.4319E+00
	STD	1.9776E+05	2.5045E+01	3.7465E+00	3.5290E+00
	FR	37%	100%	100%	100%
	Time	1.415E-01	1.5507E-01	1.4904E-01	1.511E-01
G07	mean	2.2083E+03	1.2391E+03	1.1925E+00	1.1917E+00
	STD	9.4551E+02	1.17393526e+03	3.5936E-01	3.0754E-01
	FR	0%	42%	100%	100%
	Time	1.689E-01	1.8169E-01	1.7467E-01	1.848E-01
G09	mean	1.6103E+01	5.5959E+00	1.0623E-01	1.0804E-01
	STD	4.6091E+01	5.0706E+00	4.3053E-02	5.0075E-02
	FR	100%	100%	100%	100%
	Time	1.637E-01	1.7338E-01	1.6511E-01	1.685E-01
G10	mean	7.1710E+03	6.4436E+02	6.7277E+02	5.5405E+02
	STD	3.6706E+03	1.3875E+02	1.8544E+02	1.5238E+02
	FR	87%	100%	100%	100%
	Time	1.618E-01	1.7261E-01	1.6431E-01	1.687E-01
Welded beam	mean	2.5933E-02	3.0515E-02	1.4190E-02	1.2925E-02
	STD	3.4091E-02	5.0122E-02	2.5338E-02	2.2616E-02
	FR	100%	100%	100%	100%
	Time	1.562E-01	1.7859E-01	1.7155E-01	1.747E-01
Spring design	mean	1.5033E-05	1.7876E-05	1.8845E-05	1.3696E-05
	STD	3.5099E-05	3.0631E-05	2.2837E-05	1.2960E-05
	FR	100%	100%	100%	100%
	Time	1.566E-01	1.6753E-01	1.6001E-01	1.621E-01
Speed reducer	mean	1.0543E-03	2.7143E-03	2.3643E-03	2.2943E-03
	STD	8.9235E-04	1.9231E-03	1.5829E-03	1.3334E-03
	FR	28%	100%	100%	100%
	Time	2.005E-01	1.9756E-01	1.9360E-01	2.091E-01
Pressure vessel	mean	2.4790E+02	7.2827E+01	7.3541E+01	7.9363E+01
	STD	1.0604E+02	7.7818E+01	7.5214E+01	7.9482E+01
	FR	100%	100%	100%	100%
	Time	1.665E-01	1.615E-01	1.552E-01	1.748E-01



(a) G1 problem (b) G2 problem

(c) G3 problem (d) G4 problem

(e) G6 problem (f) G7 problem

(g) G9 problem (h) G10 problem

FIGURE 3. **Mean and standard deviation of the objective value during the optimization, analytical problems (100 runs).**

to surpass other methods because the feasible region of the G2 problem accounts for 99.9975% (shown in TABLE 1); hence, the constraints are relatively easy to be satisfied.

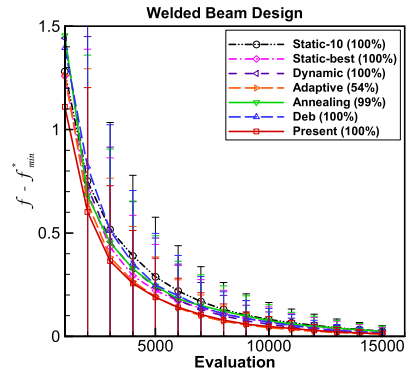
The figures also show that, for the static penalty method, better results can be obtained if the penalty factor is optimized. That is to say, the parameters of the CH method influence the optimization results significantly, and if better parameters are selected, the optimization results can be improved. However, for most real engineering problems, the optimal parameters are unknown beforehand. Despite the use of the best penalty factor, the static penalty method's performance is still not as good as the proposed method.

FIGURE 4 shows the convergence histories of the four engineering optimization problems, and similar results are obtained: the proposed method converges faster and gets a higher feasibility ratio. Like FIGURE 3, the "Static-best" in this figure also denotes the results with the best penalty factors. TABLE 4 to TABLE 9 compare the proposed method with the five existing CH methods. Note that a different population size of CS is used from TABLE 4 to TABLE 6. In these tables, the final averaged objective value (mean), standard deviation (STD), the feasibility ratio (FR), and the CPU time for a single run are shown. Note that, in these tables, there are two columns of "Static" in which the second denotes the result when the best penalty factor is used. It also can be seen that, with 15000 function evaluations, the present method always gets feasible solutions for each test problem, and still gets smaller objective value except G2 problem. It is also noted that the dynamic penalty, adaptive penalty, annealing penalty performs poorly for some problems. The reason may be that the user-defined parameters (shown in Part IV) for these methods are not well chosen since we kept the settings constant for all test cases. However, as mentioned before, the best parameters are difficult to get. TABLE 4 to TABLE 6 also indicates that the mainly user-defined setting of CS, the population size, doesn't affect the results significantly.

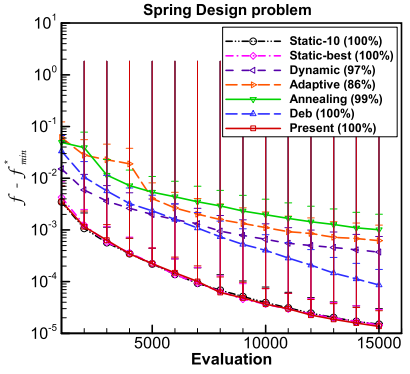
From the above comparison, we can preliminarily conclude that the present constraint-handling method outperforms the traditional methods. Consequently, the current approach is adopted to the cuckoo search that results in the constraint-handling improved cuckoo search algorithm (CICS) in this paper.

The Wilcoxon signed-rank test further demonstrates the superiority of the present method over the five other CH methods. The p-value and H are shown in TABLE 8 and TABLE 9, where H = 1 means the null hypothesis is rejected; hence the alternative hypothesis (H1) is approved. From the two tables, it can be seen that for most cases (except G2 problem) H = 1, and the present method significantly outperforms other approaches.

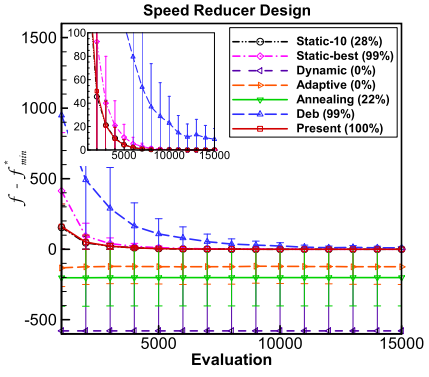
In summary, all the above results indicate a significant superiority of the proposed constraint-handling method over the other existing five methods.



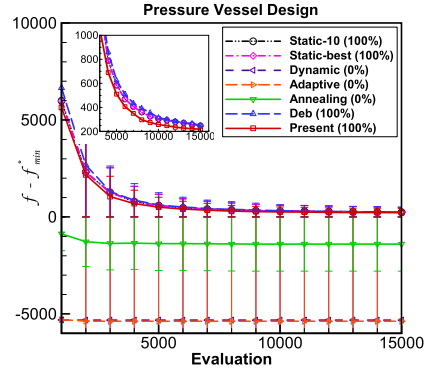
(a) Welded beam design problem



(b) Spring design problem



(c) Speed reducer design problem



(d) Pressure vessel design problem

FIGURE 4. Mean and standard deviation of the objective value during the optimization, engineering problems (100 run).

TABLE 4. Comparison of the optimization results by 6 types of constraint-handling methods, analytical problems (Cuckoo search, $n = 25$, 100 runs).

problem	type	Static (PF=10^{10})	Static (PF=best)	Dynamic	Adaptive	Annealing	Deb	Present
G01	mean	-1.6111E+01	4.4813E-01	-2.2215E+00	-2.3431E+00	-2.4045E+00	2.7380E-02	**1.7600E-02**
	STD	4.8559E+01	1.0717E+00	1.9603E+01	1.9376E+01	2.0875E+01	5.3218E-02	1.6002E-02
	FR	81%	97%	89%	77%	88%	100%	100%
	Time	1.744E-01	1.744E-01	1.803E-01	2.080E-01	2.525E-01	1.741E-01	1.794E-01
G02	mean	4.5188E-01	4.4687E-01	4.4441E-01	4.4079E-01	**4.4058E-01**	4.4606E-01	4.4445E-01
	STD	3.4273E-02	3.2791E-02	3.4483E-02	3.6793E-02	3.1750E-02	3.6844E-02	3.7344E-02
	FR	100%	100%	100%	100%	100%	100%	100%
	Time	1.973E-01	1.973E-01	1.974E-01	2.366E-01	2.794E-01	1.986E-01	2.020E-01
G03	mean	9.2309E-01	9.0229E-01	-6.2311E+04	6.8084E-01	-1.7443E+03	8.9002E-01	**8.7471E-01**
	STD	9.1959E-02	8.8036E-02	3.5645E+04	9.4817E-01	1.8262E+03	1.4034E-01	1.2623E-01
	FR	100%	96%	0%	21%	0%	100%	100%
	Time	1.491E-1	1.491E-1	1.495E-1	2.024E-1	2.372E-1	1.553E-1	1.493E-1
G04	mean	2.0900E-01	4.4900E-01	-1.4067E+03	-3.6767E+02	-6.6586E+02	6.5900E-01	**1.8857E-01**
	STD	2.0678E-01	4.3674E-01	1.6117E+02	1.9137E+01	4.3116E+02	5.4229E-01	1.8632E-01
	FR	97%	100%	0%	0%	0%	100%	100%
	Time	1.530E-01	1.530E-01	1.569E-01	1.900E-01	2.722E-01	1.697E-01	1.675E-01
G06	mean	1.1293E+05	5.4063E+01	-8.7617E+02	1.7980E+03	2.0297E+03	2.8950E+03	**2.4319E+00**
	STD	1.9776E+05	2.1274E+02	2.8835E+02	2.2550E+03	3.7377E+03	2.4183E+03	3.5290E+00
	FR	37%	68%	0%	63%	45%	66%	100%
	Time	1.415E-01	1.415E-01	1.462E-01	1.763E-01	2.254E-01	1.458E-01	1.511E-01
G07	mean	2.2083E+03	2.2250E+00	1.4989E+03	1.7344E+03	1.7341E+03	7.3961E+01	**1.1917E+00**
	STD	9.4551E+02	7.0139E-01	1.0463E+03	9.9903E+02	9.5421E+02	1.3437E+02	3.0754E-01
	FR	0%	98%	19%	0%	0%	99%	100%
	Time	1.689E-01	1.690E-01	1.737E-01	2.175E-01	1.726E-01	1.675E-01	1.848E-01
G09	mean	1.6103E+01	1.0924E+00	1.6414E+00	1.4365E+00	8.7728E+00	3.6880E+00	**1.0804E-01**
	STD	4.6091E+01	7.0656E-01	1.1023E+00	2.9107E+00	1.8456E+01	2.9866E+00	5.0075E-02
	FR	100%	100%	100%	99%	100%	100%	100%
	Time	1.637E-01	1.637E-01	1.652E-01	1.911E-01	1.798E-01	1.639E-01	1.685E-01
G10	mean	7.1710E+03	8.7122E+02	-2.6395E+03	5.5001E+03	2.0608E+03	7.6830E+03	**5.5405E+02**
	STD	3.6706E+03	2.4124E+02	1.4496E+03	4.0370E+03	1.5454E+03	2.6648E+03	1.5238E+02
	FR	87%	97%	0%	1%	25%	5%	100%
	Time	1.618E-01	1.618E-01	1.648E-01	1.726E-01	1.694E-01	1.642E-01	1.687E-01

TABLE 5. Comparison of the optimization results by 6 types of constraint-handling methods, analytical problems (Cuckoo search, $n = 20$, 100 runs).

problem	type	Static (PF=10^{10})	Static (PF=best)	Dynamic	Adaptive	Annealing	Deb	Present
G01	mean	-2.0052E+01	5.9732E-02	-8.9411E+00	-5.6294E+00	-7.1992E+00	**3.1365E-03**	3.1988E-03
	STD	5.6701E+01	1.5321E+01	2.7399E+01	2.4028E+01	2.6337E+01	**3.4532E-03**	5.7140E-03
	FR	80%	98%	81%	78%	82%	100%	100%
	Time	2.8072E-01	2.8072E-01	1.9602E-01	2.4499E-01	2.8895E-01	1.9863E-01	1.9894E-01
G02	mean	3.9414E-01	3.8811E-01	**3.8598E-01**	3.8787E-01	3.8946E-01	3.9506E-01	3.8644E-01
	STD	3.9305E-02	3.8129E-02	**3.60612E-02**	3.4984E-02	4.2478E-02	3.9282E-02	3.8078E-02
	FR	100%	100%	100%	100%	100%	100%	100%
	Time	2.1831E-01	2.1831E-01	2.1989E-01	2.6600E-01	3.0609E-01	2.1995E-01	2.2343-01
G03	mean	9.2674E-01	9.2674E-01	-4.6980E+04	4.7160E-01	-1.4150E+03	**8.6192E-01**	8.7427E-01
	STD	1.0438E-01	1.0438E-01	3.7401E+04	1.7252E+00	1.5632E+03	**1.2659E-01**	1.1963E-01
	FR	100%	100%	0%	16%	0%	100%	100%
	Time	1.4639E-01	1.4639E-01	1.4917E-01	1.8664E-01	2.3043E-01	1.4744E-01	1.4745E-01
G04	mean	4.9900E-02	1.0070E-01	-1.3797E+03	-3.7131E+02	-6.0365E+02	3.7410E-01	**3.8900E-02**
	STD	7.9216E-02	8.7739E-02	1.8620E+02	2.8201E+01	4.1733E+02	4.2877E-01	**3.9718E-02**
	FR	95%	100%	0%	0%	0%	100%	100%
	Time	1.6658E01	1.6658E01	1.6943E-01	2.1450E-01	2.5648E-01	1.6897E-01	1.7415E-01
G06	mean	1.7863E+05	8.5065E+01	-8.4894E+02	2.7223E+03	2.5184E+03	2.0003E+03	**8.2694E-01**
	STD	2.5175E+05	5.6255E+02	4.5397E+02	5.3039E+03	6.1193E+03	1.8424E+03	**1.6989E+00**
	FR	39%	51%	3%	63%	41%	89%	100%
	Time	1.5516E-01	1.5516E-01	1.5501E-01	1.9846E-01	2.3933E-01	1.5351E-01	1.5893E-01
G07	mean	2.0707E+03	1.1713E+00	1.4851E+03	1.7145E+03	1.7448E+03	1.9662E+01	**6.9277E-01**
	STD	1.0641E+03	5.0709E-01	1.0827E+03	8.8729E+02	9.0116E+02	3.6010E+01	**2.3104E-01**
	FR	0%	99%	17%	3%	0%	100%	100%
	Time	1.7799E-01	1.7799E-01	1.8668E-01	2.3148E-01	2.7249E-01	1.8145E-01	1.7934E-01
G09	mean	8.5099E+01	2.2952E+00	7.2543E-01	8.2954E-01	4.5635E+00	1.0442E+00	**4.2480E-02**
	STD	7.4738E+02	2.1535E+00	1.1964E+00	1.5655E+00	4.4503E+00	1.9347E+00	**2.1048E-02**
	FR	99%	100%	98%	88%	100%	100%	100%
	Time	1.7172E-01	1.7172E-01	1.7295E-01	2.1830E-01	2.5702E-01	1.7365E-01	1.7569E-01
G10	mean	6.2347E+03	7.2851E+02	-2.4656E+03	7.1320E+03	1.9338E+03	6.7988E+03	**3.8033E+02**
	STD	4.1353E+03	2.7721E+02	1.4919E+03	3.8816E+03	1.9379E+03	2.0629E+03	**1.0289E+02**
	FR	89%	99%	0%	3%	16%	0%	100%
	Time	1.7129E-01	1.7129E-01	1.7558E-01	2.1836E-01	2.6040E-01	1.7549E-01	1.7987E-01

TABLE 6. Comparison of the optimization results by 6 types of constraint-handling methods, analytical problems (Cuckoo search, $n = 15$, 100 runs).

problem	type	Static (PF=10^{10})	Static (PF=best)	Dynamic	Adaptive	Annealing	Deb	Present
G01	mean	-4.6807E+01	-2.9272E+00	-9.8239E+00	-1.1321E+01	-1.2194E+01	8.0204E-02	**1.1132E-03**
	STD	7.3890E+01	2.5874E+01	2.9414E+01	3.0656E+01	3.0288E+01	3.9389E-01	**8.8268E-03**
	FR	65%	84%	78%	66%	72%	100%	100%
	Time	2.0182E-01	2.0182E-01	2.4078E-01	2.5786E-01	3.0372E-01	2.0532E-01	2.1460E-01
G02	mean	3.3496E-01	**3.2971E-01**	3.3823E-01	3.3865E-01	3.3616E-01	3.3813E-01	3.3493E-01
	STD	4.6435E-02	**4.3118E-02**	4.7751E-02	4.8507E-02	4.7553E-02	4.7299E-02	5.1947E-02
	FR	100%	100%	100%	100%	100%	100%	100%
	Time	2.3340E-01	2.3340E-01	2.3553E-01	3.0760E-01	3.3299E-01	2.3700E-01	2.4181E-01
G03	mean	9.2502E-01	9.2502E-01	-4.2891E+04	7.6034E-01	-1.1196E+03	**8.5505E-01**	8.6365E-01
	STD	8.6546E-02	8.6546E-02	3.7513E+04	6.1951E-01	1.3616E+03	**1.1514E-01**	1.2797E-01
	FR	97%	97%	0%	16%	0%	100%	100%
	Time	1.6857E-01	1.6857E-01	1.6561E-01	2.1347E-01	2.6190E-01	1.6378E-01	1.6486E-01
G04	mean	4.1000E-03	1.1700E-02	-1.3477E+03	-3.5668E+02	-5.4500E+02	-2.4970E-01	**2.8000E-03**
	STD	1.5999E-02	3.4437E-02	2.3565E+02	5.7965E+01	4.3139E+02	6.1222E+00	**4.3074E-03**
	FR	46%	100%	0%	0%	0%	82%	100%
	Time	1.8508E-01	1.8508E-01	1.8730E-01	2.3334E-01	2.8244E-01	1.8274E-01	1.9250E-01
G06	mean	1.3789E+05	1.8861E+04	-7.8507E+02	2.8711E+03	2.3823E+03	3.8668E+02	**9.8610E-02**
	STD	2.2800E+05	8.8859E+04	3.6120E+02	5.7593E+03	8.2159E+03	7.5738E+02	**2.1054E-01**
	FR	36%	83%	0%	60%	51%	99%	100%
	Time	1.6818E-01	1.6818E-01	1.7025E-01	2.2192E-01	2.6719E-01	1.7089E-01	1.7745E-01
G07	mean	2.1759E+03	5.5899E-01	1.6580E+03	1.6435E+03	1.6707E+03	2.8603E+00	**4.0702E-01**
	STD	1.0730E+03	3.1415E-01	9.2480E+02	1.0797E+03	9.2459E+02	3.5845E+00	**2.2591E-01**
	FR	0%	94%	6%	2%	0%	100%	100%
	Time	1.9810E-01	1.9810E-01	2.0245E-01	2.5257E-01	2.9639E-01	1.9609E-01	2.0542E-01
G09	mean	6.4128E+00	1.0588E+00	2.5104E-01	6.3942E-01	2.1503E+00	6.6215E-02	**1.4544E-02**
	STD	1.1511E+01	3.2237E+00	4.3238E-01	2.0321E+00	2.8462E+00	7.0253E-02	**1.1110E-02**
	FR	100%	100%	100%	83%	100%	100%	100%
	Time	1.8668E-01	1.8668E-01	1.9638E-01	2.4157E-01	2.8379E-01	2.1514E-01	2.0249E-01
G10	mean	5.3545E+03	4.5415E+02	-1.8927E+03	6.6170E+03	1.4306E+03	4.6099E+03	**2.2248E+02**
	STD	4.1379E+03	3.0597E+02	1.7559E+03	4.2257E+03	1.8354E+03	1.5700E+03	**9.7345E+01**
	FR	89%	99%	0%	3%	30%	3%	100%
	Time	4.1379E+03	4.1379E+03	1.9384E-01	2.4391E-01	2.9160E-01	1.9619E-01	2.0050E-01

TABLE 7. Comparison of the optimization results by 6 types of constraint-handling methods, engineering problems (Cuckoo search, 100 runs).

problem	type	Static (PF=10^{10})	Static (PF=best)	Dynamic	Adaptive	Annealing	Deb	Present
Welded beam	mean	2.5933E-02	2.1883E-02	1.4968E-02	1.1123E-02	2.4689E-02	1.7029E-02	**1.2925E-02**
	STD	3.4091E-02	4.6095E-02	2.1509E-02	3.9641E-02	4.5128E-02	2.3058E-02	2.2616E-02
	FR	100%	100%	100%	54%	99%	100%	100%
	Time	1.562E-01	1.562E-01	1.602E-01	2.109E-01	2.477E-01	1.674E-01	1.747E-01
Spring design	mean	1.5033E-05	1.4557E-05	3.7404E-04	6.2097E-04	1.01070E-03	8.6096E-05	**1.3696E-05**
	STD	3.5099E-05	2.6077E-05	7.8035E-04	1.6588-03	1.0295E-03	8.8123E-05	1.2960E-05
	FR	100%	100%	97%	86%	99%	100%	100%
	Time	1.566E-01	1.566E-01	1.586E-01	1.856E-01	1.777E-01	1.551E-01	1.621E-01
Speed reducer	mean	1.0543E-03	1.3704E-02	-5.7829E+02	-1.2469E+02	-2.0102E+02	9.2169E+00	**2.2943E-03**
	STD	8.9235E-04	1.0652E-02	4.1746E+01	1.3658E+02	1.2927E+02	5.4890E+01	1.3334E-03
	FR	28%	99%	0%	22%	0%	99%	100%
	Time	2.005E-01	2.005E-01	2.113E-01	2.204E-01	2.977E-01	2.091E-01	2.091E-01
Pressure vessel	mean	2.4790E+02	2.4790E+02	-5.3187E+03	-5.3778E+03	-1.3970E+03	2.5242E+02	**7.9363E+01**
	STD	1.0604E+02	1.0604E+02	7.4035E+01	2.6562E+01	1.7763E+03	1.0614E+02	7.9482E+01
	FR	100%	100%	0%	0%	0%	100%	100%
	Time	1.665E-01	1.665E-01	1.609E-01	1.867E-01	2.437E-01	1.623E-01	1.748E-01

2) APPLICATION OF PARTICLE SWARM OPTIMIZATION

To further demonstrate the proposed constraint-handling method, it is also applied to PSO and compared to the existing constraint-handling methods. TABLE 10 and TABLE 11 show the feasibility ratio, averaged objective value, standard deviation, and the CPU time, obtained by the present CH method and the five-existing constraint-handling methods. One can see that the current algorithm always finds feasible solutions, and in most cases, it gets better solutions than the other methods.

TABLE 12 and TABLE 13 show the Wilcoxon signed-rank test results for the proposed method versus the other methods. They also indicate that the present method significantly outperforms the other methods when applied to particle swarm optimization algorithm.

TABLE 8. Wilcoxon signed-ranks tests of different types of constraint-handling methods, analytical problems (cuckoo search, $H = 1$ means the proposed method significantly outperforms the other method with $\alpha = 0.1$).

Comparison	G01		G02		G03		G04		G06		G07		G09		G10	
	p-value	*H*	*p*-value	*H*	*p*-value	*H*	*p*-value	*H*	*p*-value	*H*	*p*-value	*H*	*p*-value	*H*	*p*-value	*H*
Present vs. Static	0.000000	**1**	0.060695	**1**	0.000384	**1**	0.137868	0	0.000000	**1**	0.000000	**1**	0.000000	**1**	0.000000	**1**
Present vs. Dynamic	0.000000	**1**	0.481489	0	0.000000	**1**	0.000000	**1**	0.000000	**1**	0.000000	**1**	0.000000	**1**	0.000000	**1**
Present vs. Adaptive	0.000000	**1**	0.821149	0	0.000000	**1**	0.000000	**1**	0.000000	**1**	0.000000	**1**	0.000000	**1**	0.000000	**1**
Present vs. Annealing	0.000000	**1**	0.861753	0	0.000000	**1**	0.000000	**1**	0.000000	**1**	0.000000	**1**	0.000000	**1**	0.000000	**1**
Present vs. Deb's	0.054359	**1**	0.384388	0	0.056671	**1**	0.000000	**1**	0.000000	**1**	0.000000	**1**	0.000000	**1**	0.000000	**1**

TABLE 9. Wilcoxon signed-ranks tests of different types of constraint-handling methods, engineering problems (cuckoo search, $H = 1$ means the proposed method significantly outperforms the other method with $\alpha = 0.1$).

Comparison	Welded beam design problem		Spring design problem		Speed reducer design problem		Pressure vessel design problem	
	p-value	*H*	*p*-value	*H*	*p*-value	*H*	*p*-value	*H*
Present vs. Static	0.000000	**1**	0.345637	0	0.000000	**1**	0.007354	**1**
Present vs. Dynamic	0.052491	**1**	0.000000	**1**	0.000000	**1**	0.000000	**1**
Present vs. Adaptive	0.000000	**1**	0.000000	**1**	0.000000	**1**	0.000000	**1**
Present vs. Annealing	0.001205	**1**	0.000000	**1**	0.000000	**1**	0.000000	**1**
Present vs. Deb's	0.005341	**1**	0.000000	**1**	0.000000	**1**	0.000176	**1**

TABLE 10. Comparison of the optimization results by 6 types of constraint-handling methods, analytical problems (PSO, 100 runs).

problem	type	Static (PF=10^{10})	Static (PF=best)	Dynamic	Adaptive	Annealing	Deb	Present
G01	mean	2.4602E+00	2.3392E+00	-1.7200E+01	2.2626E+00	-3.0808E+01	2.5009E+00	**2.2616E+00**
	STD	1.4468E+00	1.6110E+00	2.8876E+01	1.3266E+00	1.9895E+01	1.6956E+00	1.4955E+00
	FR	100%	100%	56%	3%	11%	100%	100%
	Time	0.449	0.449	0.446	0.498	0.542	0.462	0.512
G02	mean	3.3607E-01	**3.2392E-01**	3.4204E-01	3.4523E-01	3.2742E-01	3.4856E-01	3.3010E-01
	STD	8.2322E-02	-4.7414E-01	7.5870E-02	8.7321E-02	7.9406E-02	8.1994E-02	9.3469E-02
	FR	100%	100%	100%	100%	100%	100%	100%
	Time	0.472	0.472	0.480	0.526	0.571	0.480	0.513
G03	mean	7.8388E-01	8.3218E-01	-7.0407E+04	-1.2697E-02	-1.4343E+03	-1.4311E+03	**8.0378E-01**
	STD	1.5814e-01	1.3772e-01	1.0657e+04	6.7003e-05	1.1283e+03	1.5471e+03	1.5297e-01
	FR	91%	100%	0%	0%	0%	0%	100%
	Time	0.507	0.508	0.495	0.560	0.571	0.484	0.527
G04	mean	-2.0000E-04	1.9900E-02	-1.5317E+03	-3.7088E+02	-6.1642E+02	-4.8270E+02	**0.0000E+00**
	STD	1.5503E-03	1.9831E-01	3.7839E+01	1.9009E-05	3.5549E+02	4.0813E+02	1.1098E-02
	FR	3%	99%	0%	0%	0%	1%	100%
	Time	0.464	0.464	0.465	0.468	0.564	0.464	0.510
G06	mean	1.0572E+01	1.2335E+01	-9.7999E+02	-9.9014E+02	2.3346E+03	1.2649E+02	**9.6238E+00**
	STD	8.401E+00	1.655E+01	9.977E+01	2.653E-08	6.223E+03	8.202E+02	8.697E+00
	FR	97%	100%	0%	0%	38%	96%	100%
	Time	0.451	0.451	0.446	0.463	0.568	0.521	0.511
G07	mean	1.4565E+00	**1.3943E+00**	7.5958E+02	1.3460E+00	1.2232E+03	8.1906E+01	1.4369E+00
	STD	1.1518E+00	9.6165E-01	8.0234E+02	1.0090E+00	7.5529E+02	2.8716E+02	1.1529E+00
	FR	100%	100%	32%	7%	11%	92%	100%
	Time	0.521	0.521	0.525	0.490	0.589	0.479	0.521
G09	mean	**1.5829E-01**	**1.5829E-01**	1.6282E-01	1.4728E-01	1.5973E-01	1.6268E+01	1.6362E-01
	STD	9.9874E-02	9.9874E-02	1.2464E-01	9.8745E-02	1.1989E-01	1.6121E+02	1.3532E-01
	FR	100%	100%	66%	14%	76%	97%	100%
	Time	0.472	0.472	0.481	0.491	0.581	0.534	0.544
G10	mean	6.6478E+02	6.0152E+02	-4.4455E+03	-4.5758E+03	1.9698E+03	2.8865E+03	**5.5592E+02**
	STD	4.7155E+02	4.0029E+02	5.0183E+02	6.1774E+01	2.2167E+03	2.7008E+03	3.4804E+02
	FR	89%	100%	0%	0%	0%	0%	100%
	Time	0.493	0.493	0.489	0.480	0.593	0.541	0.493

TABLE 11. Comparison of the optimization results by 6 types of constraint-handling methods, engineering problems (PSO, 100 runs).

problem	type	Static (PF=10^{10})	Static (PF=best)	Dynamic	Adaptive	Annealing	Deb	Present
Welded beam	mean	4.2375E-03	2.0395E-03	4.6478E-03	1.5885E-04	2.6750E-03	4.6533E-03	**4.5240E-03**
	STD	1.5935E-02	6.3094E-03	1.5481E-02	1.0151E-02	1.0480E-02	2.6429E-02	1.7827E-02
	FR	81%	99%	61%	1%	39%	100%	100%
	Time	0.495	0.495	0.504	0.490	0.612	0.560	0.605
Spring design	mean	6.0969E-04	7.5401E-04	8.4985E-04	6.6482E-04	8.3674E-04	**6.1879E-04**	7.8075E-04
	STD	8.2646E-04	1.0864E-03	7.9945E-04	8.3915E-04	7.5646E-04	7.3628E-04	9.7329E-04
	FR	73%	99%	28%	8%	33%	100%	100%
	Time	0.481	0.481	0.488	0.475	0.597	0.492	0.536
Speed reducer	mean	-1.6568E-04	**4.3200E-06**	-6.0653E+02	-5.0270E+02	-2.6073E+02	-6.4482E+01	**4.3200E-06**
	STD	1.7367E-08	3.2799E-06	2.1860E+01	3.8376E-06	1.0759E+02	1.4244E+02	2.0039E-06
	FR	0%	100%	0%	0%	0%	1%	100%
	Time	0.534	0.534	0.526	0.515	0.621	0.525	0.582
Pressure vessel	mean	4.5567E+02	4.4351E+02	-5.3669E+03	-5.3853E+03	-4.1065E+03	2.7941E+02	**4.3412E+02**
	STD	2.3452E+02	2.2750E+02	3.3371E+01	1.2563E-12	1.2796E+03	8.6352E+02	2.4252E+02
	FR	2%	100%	0%	0%	0%	88%	100%
	Time	0.493	0.493	0.507	0.495	0.593	0.508	0.551

TABLE 12. Wilcoxon signed-ranks tests of different types of constraint-handling methods, analytical problems (PSO, $\alpha = 0.1$).

Comparison	G01		G02		G03		G04		G06		G07		G09		G10	
	p-value	H	p-value	H	p-value	H	p-value	H	p-value	H	p-value	H	p-value	H	p-value	H
Present vs. Static	0.210039	0	0.316963	0	0.309652	0	0.000000	**1**	0.012997	**1**	0.443267	0	0.477379	0	0.011265	**1**
Present vs. Dynamic	0.000000	**1**	0.213024	0	0.000000	**1**	0.000000	**1**	0.000000	**1**	0.000000	**1**	0.000000	**1**	0.000000	**1**
Present vs. Adaptive	0.000000	**1**	0.110144	0	0.000000	**1**	0.000000	**1**	0.000000	**1**	0.000000	**1**	0.000000	**1**	0.000000	**1**
Present vs. Annealing	0.000000	**1**	0.681811	0	0.000000	**1**	0.000000	**1**	0.000000	**1**	0.000000	**1**	0.000099	**1**	0.000000	**1**
Present vs. Deb's	0.300019	0	0.106310	0	0.000000	**1**	0.000000	**1**	0.000000	**1**	0.001511	**1**	0.134489	0	0.000000	**1**

TABLE 13. Wilcoxon signed-ranks tests of different types of constraint-handling methods, engineering problems (PSO, $\alpha = 0.1$).

Comparison	Welded beam design problem		Spring design problem		Speed reducer design problem		Pressure vessel design problem	
	p-value	H	p-value	H	p-value	H	p-value	H
Present vs. Static	0.008393	**1**	0.000013	**1**	0.000000	**1**	0.000000	**1**
Present vs. Dynamic	0.000000	**1**	0.000000	**1**	0.000000	**1**	0.000000	**1**
Present vs. Adaptive	0.000000	**1**	0.000000	**1**	0.000000	**1**	0.000000	**1**
Present vs. Annealing	0.000000	**1**	0.000000	**1**	0.000000	**1**	0.000000	**1**
Present vs. Deb's	0.005341	**1**	0.000000	**1**	0.000000	**1**	0.000581	**1**

3) APPLICATION OF GENETIC ALGORITHM

Finally, the proposed constraint-handling method is applied to the genetic algorithm and compared with the existing methods. TABLE 14 and TABLE 15 show the feasibility ratio, averaged objective value, standard deviation, and the CPU time, obtained by the proposed method and the five existing methods. TABLE 16 and TABLE 17 show the results of Wilcoxon signed ranks tests. These tables indicate that the proposed method significantly outperforms the dynamic penalty, adaptive penalty, and annealing penalty for most cases. In some cases, the present method performs close to the static penalty and Deb's approach, while in other cases, the former method performs significantly better. To sum up, the proposed constraint-handling method performs comparable or better than the other techniques when applied to a genetic algorithm.

4) COMPARISON OF THE CICS WITH RECENT OPTIMIZERS

This section compares the CICS with some other metaheuristic algorithms, including the two well-known traditional algorithms and six most recent algorithms, to validate the effect of the proposed CICS algorithm. In last years,

TABLE 14. Comparison of the optimization results by 6 types of constraint-handling methods, analytical problems (GA, 100 runs).

problem	type	Static (PF=10^{10})	Static (PF=best)	Dynamic	Adaptive	Annealing	Deb	Present
G01	mean	9.2229E-02	**9.2019E-02**	1.3714E-01	1.8179E-01	-1.9525E+00	1.5243E-01	1.4068E-01
	STD	4.2260E-01	4.2333E-01	5.7120E-01	6.6458E-01	8.8789E+00	5.8646E-01	5.2163E-01
	FR	100%	100%	73%	73%	81%	100%	100%
	Time	0.520	0.520	0.516	0.556	0.586	0.522	0.538
G02	mean	1.1477E-01	1.1477E-01	1.1644E-01	-4.5084E+02	**1.1284E-01**	1.1469E-01	1.1509E-01
	STD	4.3589E-02	4.3589E-02	4.8889E-02	6.1221E+02	3.8317E-02	4.2092E-02	4.2734E-02
	FR	100	100	98	1	100	100	100
	Time	0.630	0.630	0.632	0.681	0.709	0.642	0.659
G03	mean	9.0631E-01	9.2992E-01	-3.5640E+04	7.8357E-01	-2.5278E+03	-2.6674E+03	**9.0798E-01**
	STD	1.3112E-01	8.9100E-02	1.3541E+04	1.5329E-01	1.5865E+03	2.7391E+03	1.1495E-01
	FR	76%	100%	0%	0%	0%	0%	100%
	Time	0.621	0.621	0.598	0.678	0.694	0.658	0.672
G04	mean	1.8751E+02	1.9190E+02	-8.7353E+02	-2.8210E+02	-9.2503E+02	-7.9838E+02	**1.6485E+02**
	STD	8.1874E+01	1.3161E+02	2.4068E+02	1.0984E+02	2.4912E+02	2.9560E+02	8.4177E+01
	FR	54%	100%	0%	0%	0%	0%	100%
	Time	0.500	0.501	0.503	0.548	0.571	0.508	0.524
G06	mean	9.5572E+02	**8.8589E+02**	-9.8711E+02	6.9033E+02	-3.3872E+02	4.6410E+02	9.3783E+02
	STD	9.0529E+02	5.0594E+02	3.0527E+01	6.3181E+02	9.3736E+02	7.8164E+02	7.4298E+02
	FR	97	99	0	81	28	68	98
	Time	0.572	0.572	0.573	0.636	0.649	0.587	0.606
G07	mean	3.7108E+00	3.7108E+00	3.0511E+00	3.3257E+00	1.6909E+02	3.7234E+00	**3.2983E+00**
	STD	3.3499E+00	3.3499E+00	3.1211E+00	3.5919E+00	3.4218E+02	4.0696E+00	3.0558E+00
	FR	100%	100%	47%	91%	80%	100%	100%
	Time	0.493	0.493	0.496	0.552	0.572	0.511	0.526
G09	mean	1.8455E+00	1.9929E+00	1.8887E+00	1.9210E+00	1.5051E+02	**1.7544E+00**	1.7949E+00
	STD	3.2978E+00	3.3545E+00	3.6880E+00	3.0316E+00	1.1743E+03	2.0151E+00	2.4788E+00
	FR	90%	100%	64%	91%	96%	100%	100%
	Time	0.586	0.586	0.609	0.639	0.665	0.601	0.629
G10	mean	3.8439E+03	2.8793E+03	-4.4500E+03	1.3099E+03	-1.5592E+03	-9.6301E+02	**1.2067E+03**
	STD	4.1475E+03	2.7682E+03	3.3602E+02	2.5591E+03	1.6078E+03	1.5916E+03	8.0908E+02
	FR	90%	99%	0%	4%	3%	8%	100%
	Time	0.484	0.484	0.488	0.536	0.563	0.510	0.521

TABLE 15. Comparison of the optimization results by 6 types of constraint-handling methods, engineering problems (GA, 100 runs).

problem	type	Static (PF=10^{10})	Static (PF=best)	Dynamic	Adaptive	Annealing	Deb	Present
Welded beam	mean	4.0019E-01	3.8025E-01	3.0598E-01	4.1391E-01	**3.5467E-01**	4.0331E-01	4.2866E-01
	STD	2.7444E-01	2.5903E-01	2.4281E-01	2.9201E-01	2.9042E-01	2.9681E-01	3.1105E-01
	FR	100%	100%	92%	95%	100%	100%	100%
	Time	0.495	0.495	0.497	0.547	0.575	0.511	0.528
Spring design	mean	7.6002E-04	6.4904E-04	9.4314E-04	8.8849E-04	1.2236E-03	**6.0515E-04**	6.4519E-04
	STD	8.5943E-04	5.7995E-04	8.9161E-04	9.2187E-04	1.0986E-03	5.6942E-04	6.9356E-04
	FR	92%	100%	31%	19%	52%	100%	100%
	Time	0.589	0.589	0.585	0.634	0.661	0.596	0.623
Speed reducer	mean	1.3543E-03	6.0643E-03	-2.3504E+02	-2.0339E+02	-2.3544E+02	-1.9065E+02	**1.9643E-03**
	STD	4.1512E-03	2.5300E-02	8.6472E+01	1.1205E+02	9.5859E+01	9.1251E+01	7.8139E-03
	FR	42%	100%	0%	0%	0%	2%	100%
	Time	0.531	0.531	0.522	0.573	0.560	0.535	0.553
Pressure vessel	mean	9.1592E+02	8.9039E+02	-5.3506E+03	-5.3605E+03	-1.6027E+03	-4.1516E+02	**8.8958E+02**
	STD	3.6018E+02	3.6124E+02	3.6124E+02	9.1083E+01	2.0148E+03	2.0071E+03	3.5928E+02
	FR	100%	100%	0%	0%	1%	57%	100%
	Time	0.583	0.583	0.588	0.633	0.663	0.596	0.618

a large number of meta-heuristic optimization algorithms have been proposed by the inspiration of certain natural phenomena, for example, the firefly algorithm (FA) [58], grey wolf optimizer (GWO) [59], moth-flame optimization (MFO) [16], salp swarm algorithm (SSA) [17], artificial bee colony [60], tree-seed algorithm [10] and so on. This research introduces the improved PSO [3], improved GA [48], MFO [16], SSA [17], modified ABC [60], modified TSA [10], weighted differential evolution (WDE) [61], and two modified particle swarm optimization MPSO and TACPSO [62] to compare with the presented CICS algorithm.

Following the comparison of CH methods in the previous sections, the proposed CH method and the five existing CH methods are first applied to the different meta-heuristic algorithms. The best one out of the different CH methods is used for comparison. Actually, for most problems, the newly proposed constraint-handling method performs best, which has been shown in the previous sections. Each optimization

TABLE 16. Wilcoxon signed-ranks tests of different types of constraint-handling methods, analytical problems (GA, $\alpha = 0.1$).

Comparison	G01		G02		G03		G04		G06		G07		G09		G10	
	p-value	H	p-value	H	p-value	H	p-value	H	p-value	H	p-value	H	p-value	H	p-value	H
Present vs. Static	0.435133	0	0.366133	0	0.000029	**1**	0.000000	**1**	0.163138	0	0.181560	0	0.631277	0	0.000000	**1**
Present vs. Dynamic	0.000000	**1**	0.291704	0	0.000000	**1**	0.000000	**1**	0.000000	**1**	0.000000	**1**	0.000000	**1**	0.000000	**1**
Present vs. Adaptive	0.000018	**1**	0.000000	**1**	0.000000	**1**	0.000000	**1**	0.000649	**1**	0.089690	**1**	0.072204	**1**	0.000000	**1**
Present vs. Annealing	0.000000	**1**	0.792922	0	0.000000	**1**	0.000000	**1**	0.000000	**1**	0.000000	**1**	0.136735	0	0.000000	**1**
Present vs. Deb's	0.292886	0	0.459604	0	0.000000	**1**	0.000000	**1**	0.000011	**1**	0.278859	0	0.282335	0	0.000000	**1**

TABLE 17. Wilcoxon signed-ranks tests of different types of constraint-handling methods, engineering problems (GA, $\alpha = 0.1$).

Comparison	Welded beam design problem		Spring design problem		Speed reducer design problem		Pressure vessel design problem	
	p-value	H	p-value	H	p-value	H	p-value	H
Present vs. Static	0.764339	0	0.016025	**1**	0.000000	**1**	0.340580	0
Present vs. Dynamic	0.801655	0	0.000000	**1**	0.000000	**1**	0.000000	**1**
Present vs. Adaptive	0.288174	0	0.000000	**1**	0.000000	**1**	0.000000	**1**
Present vs. Annealing	0.937211	0	0.000000	**1**	0.000000	**1**	0.000000	**1**
Present vs. Deb's	0.751453	0	0.653094	0	0.000000	**1**	0.000000	**1**

case is repeated 100 times independently, and the statistical results, i.e., mean ($\bar{f}$), standard deviations (σ), and the CPU time, are compared in TABLE 18. Besides, to demonstrate the significant superiority of the CICS to the other algorithms, the Wilcoxon signed-rank test is also employed, and the p-value is shown in this table. Note that the p-value in the column of GA denotes the p-value obtained when comparing the CICS and GA, so is the PSO, MFO, SSA, ABC, TSA, WDE, MPSO, and TACPSO. The smaller the p-value represents, the higher the superiority of the CICS algorithm. TABLE 18 shows that, for the mean and standard deviation of the objective value, the proposed CICS performs best among the ten algorithms. The CICS obtained the smallest averaged-objective value and standard deviation for 11 out of the 14 problems. Besides, most of the p-value is very small, which represents the significant superiority of the CICS to the other algorithms.

V. APPLICATION IN AIRFOIL AERODYNAMIC SHAPE OPTIMIZATION

Meta-heuristic algorithms have been applied to airfoil optimization problems. In this section, the CH improved cuckoo search algorithm (CICS) is applied to an airfoil aerodynamic shape optimization problem and compared to the original cuckoo search (CS).

The problem is to minimize the drag coefficient of the RAE2822 airfoil at the freestream of Mach number 0.734, lift coefficient 0.824, and Reynolds number 6.5×10^6, subject to a cross-section area and pithing moment constraint. In summary, the optimization problem is

$$\begin{aligned} &Minimize: C_d \\ &subjectto: \ (1)\ C_l = 0.824 \\ &\qquad\qquad (2)\ C_m \geq C_{m0} \\ &\qquad\qquad (3)\ Area \geq Area_0 \end{aligned} \tag{19}$$

where C_{m0} and $Area_0$ are the pitching moment and area of the initial airfoil.

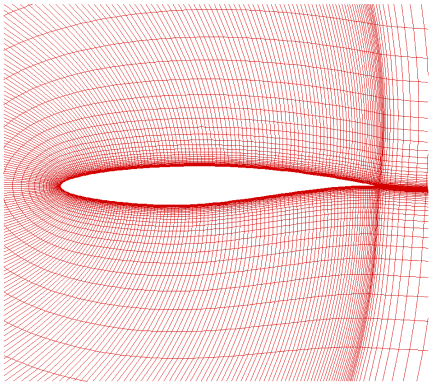


FIGURE 5. Sketch of "C" grid for airfoil flow simulation.

The flow analyses are performed with an in-house code called PMNS2D [6]. It solves the Reynolds-averaged Navier-Stokes (RANS) equations to simulate the flow around an

TABLE 18. Comparison of the CICS to benchmark algorithms (the smaller the *p*-value, the greater the advantage of CICS).

problem	type	CICS	GA	PSO	MFO	SSA	ABC	TSA	WDE	MPSO	TACPSO
G01	mean	-15.00000	-14.891068	-14.804275	-13.360000	-9.222719	-14.999987	**-15.000000**	-14.978844	-8.040000	-13.714531
	STD	6.208E-9	0.538	0.708	1.888	1.449	9.274E-5	**2.230E-15**	1.046E-2	2.049	2.200
	p	/	0.00000	0.00000	0.00004	0.00000	0.92195	1.00000	0.00000	0.00000	0.26969
	Time	0.64315	2.95988	1.77631	0.43096	0.50135	0.90927	0.40897	0.89022	0.43247	0.42430
G02	mean	-0.570215	**-0.6995769**	-0.683705	-0.4602898	-0.3581364	-0.4836021	-0.5621982	-0.6497581	-0.5444269	-0.6247996
	STD	0.0696	**0.039**	0.044	0.128	0.063	0.047	0.064	0.025	0.117	0.007
	p	/	1.00000	1.00000	0.00000	0.00000	0.00000	0.09194	1.00000	0.01760	0.99999
	Time	0.73524	3.07540	1.91023	0.52788	0.61770	0.98509	0.50161	1.03393	0.52864	0.50618
G03	mean	**-0.214440**	-0.073984	-0.088071	-0.009733	-0.1640897	-0.1247803	-0.0275010	-0.0949733	-7.155148E-5	-6.536037E-2
	STD	**0.1433**	9.475E-2	0.118	5.856E-02	0.206	1.0563E-1	5.279E-2	6.929E-2	7.155E-4	0.138
	p	/	0.00000	0.00000	0.00000	0.09194	0.00155	0.00000	0.00000	0.00000	0.00000
	Time	0.49569	3.08834	1.64588	0.31213	0.38108	0.70642	0.28904	0.74489	0.31663	0.30797
G04	mean	**-30665.538**	-30427.837	-30455.5295	-30661.658	-30200.679	-30214.475	-30663.060	-30593.878	-30665.532	-30665.526
	STD	**3.735E-3**	164.294	153.445	14.411	286.310	153.482	1.870	40.646	4.599E-2	6.263E-2
	p	/	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	1.00000	1.00000
	Time	0.12889	0.60133	0.35213	0.077192	0.09326	0.18332	0.07186	0.15447	0.07258	0.07292
G05	mean	**5303.68852**	5962.01454	6223.45218	5762.21989	5564.72594	5350.08357	5400.11394	5407.75806	5583.22973	5530.68434
	STD	**190.035**	665.572	617.05	500.137	352.792	269.760	343.035	289.973	432.016	405.108
	p	/	0.00000	0.00000	0.00000	0.00000	0.0000	0.00000	0.00000	0.00000	0.00009
	Time	0.59860	2.88180	1.76703	0.39816	0.47412	0.83441	0.36271	0.77408	0.38139	0.38008
G06	mean	**-6961.81388**	-5831.08057	-5825.57621	-6710.37295	-6906.02904	-6512.31494	-6875.61957	-6898.85954	-6193.27449	-6961.81386
	STD	**7.503E-9**	694.286	730.272	696.017	199.891	257.738	56.479	67.057	270.124	216.666
	p	/	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000
	Time	0.23956	1.17743	0.68277	0.13695	0.17253	0.31683	0.06932	0.28700	0.13523	0.13337
G07	mean	**24.30693**	24.4768922	29.483965	96.108495	58.298596	35.375419	27.888239	28.032155	222.247519	29.871558
	STD	**1.118E-3**	44.313	6.691	137.216	150.453	6.708	1.000	1.331	457.033	14.091
	p	/	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000
	Time	0.24396	1.18665	0.70083	0.17064	0.20188	0.33714	0.08001	0.34392	0.166218	0.163349
G08	mean	-9.5825E-2	-9.5825E-2	-9.5825E-2	-9.5825E-2	-9.5825E-02	-9.5039E-2	-9.5825E-2	-9.5825E-2	-9.5825E-2	**-9.5825E-2**
	STD	2.069E-11	5.493E-13	1.803E-5	5.311E-17	3.456E-13	6.673E-3	1.079E-9	1.178E-6	5.235E-17	**5.125E-17**
	p	/	1.00000	0.00000	1.00000	0.00000	0.00000	0.00038	0.00000	1.00000	1.00000
	Time	0.04973	0.23622	0.14290	0.03094	0.03727	0.06347	0.02934	0.05879	0.02889	0.02697
G09	mean	**680.630384**	682.399274	682.072822	681.290497	681.899030	685.980766	681.400295	681.318672	680.838602	680.841883
	STD	**1.485E-4**	3.415	1.885	0..582	0.951	2.287	0.316	0.293	0.303	0.173
	p	/	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000
	Time	0.23126	1.14388	0.71088	0.16041	0.19536	0.33020	0.07926	0.32397	0.15449	0.15642
G10	mean	**7049.85062**	8108.46289	8057.51423	10672.8035	8176.29773	8281.992134	7736.36891	7282.944457	10443.8785	7975.573300
	STD	**1.108**	765.137	673.084	2926.949	1738.645	522.146	204.757	98.823	3010.199	691.230
	p	/	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000
	Time	0.58073	2.96543	1.70444	0.39581	0.49018	0.77895	0.36735	0.808213	0.40040	0.38721
Welded beam	mean	**1.724858**	2.197385	2.13699306	1.8239816	1.996919	1.988011	1.815234	1.762432	1.751176	1.777892
	STD	**5.515E-6**	0.394	0.322	0.217	0.160	0.123	6.841E-2	3.252E-2	0.102	0.139
	p	/	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00163	0.00000
	Time	0.23277	1.16885	0.78161	0.16170	0.19527	0.33269	0.08091	0.31528	0.16075	0.16063
Spring design	mean	**0.0126656**	0.01338304	0.0133556	0.0130985	0.0129213	0.0128592	0.0127510	0.0126752	0.0130445	0.0127223
	STD	**1.518E-6**	6.510E-4	5.932E-4	8.464E-4	3.930E-04	9.534E-5	8.742E-5	1.869E-5	1.791E-3	8.222E-5
	p	/	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000
	Time	0.55234	2.85466	1.74689	0.37083	0.47018	0.78781	0.35385	0.76674	0.39164	0.39201
Speed reducer	mean	**2994.34132**	2994.34204	2994.34192	2997.20988	3042.42924	2994.34174	2994.341709	2994.505514	3069.84493	3033.97637
	STD	**3.983E-8**	2.286E-3	3.720E-4	9.582	30.273	2.336E-3	1.716E-4	6.538E-2	63.596	46.093
	p	/	0.00000	0.00000	1.00000	0.00000	1.00000	0.00000	0.00000	0.00000	0.00000
	Time	0.29728	1.19468	0.81396	0.95397	0.22807	0.36530	0.09621	0.35684	0.19181	0.18964
Pressure vessel	mean	**5885.33262**	6883.67848	6634.86182	6253.09732	6235.97217	6031.79048	5958.32519	5889.40923	6101.27914	6269.18037
	STD	**1.097E-4**	406.266	399.030	436.993	259.808	62.791	85.034	8.634	402.078	352.956
	p	/	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000
	Time	0.54815	2.90488	1.68849	0.35616	0.45636	0.77311	0.34919	0.76442	0.38379	0.37621

airfoil. The equations are solved using the cell-centered finite volume method on structured grids of "C" topology. The Jameson's second-order center scheme is introduced to discretize the volumes, and the Apalart-Allmaras one-equation turbulent model is used for turbulence closure. To improve the convergence speed, the implicit residual

smoothing, local-time stepping, and multigrid techniques, are used. FIGURE 5 shows the topology of the "C" type grid with the distribution of 321 × 65 used in this research, which is generated automatically and fast by an in-house code. FIGURE 6 compares the pressure distribution (Cp) of the RAE2822 between the computational results by PMNS2D and the wind-tunnel experimental results [63] at the operating condition of $Ma = 0.73, \mathrm{Re} = 6.5 \times 10^6, \alpha = 2.79°$, in which x/c denotes the abscissa of the airfoil. As one can see, the computational results agree well with the experimental data. Hence, the aerodynamic performance evaluation method used in this research is effective and can be used for aerodynamic optimization design.

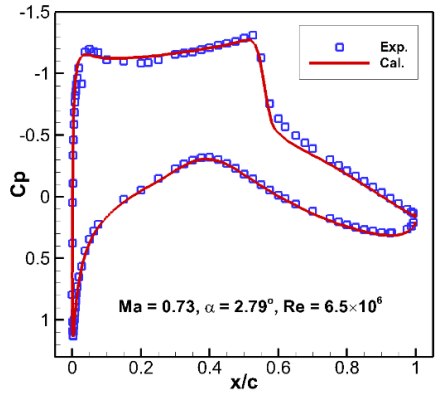


FIGURE 6. Comparison of pressure distributions between computed and experimental data for RAE2822.

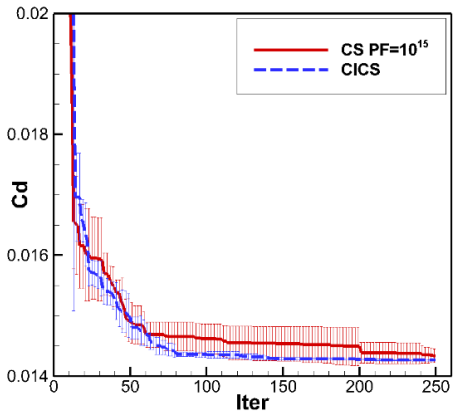


FIGURE 7. Mean and standard deviation of the drag coefficient of the repeated 30 runs during the optimization (airfoil design problem).

The CST (class function/shape function transformation) method proposed by Kulfan [64] is introduced to parameterize the shape of an airfoil since it requires a small number of design variables to describe an airfoil. Here the 4th order Bernstein polynomial is adopted; hence there are ten design variables for this design optimization problem.

The problem is solved by CICS and CS for 30 times, respectively, and the statistical results are calculated. For

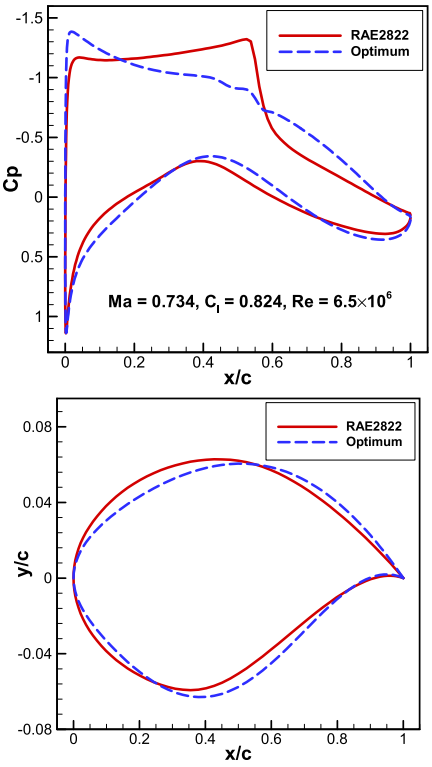


FIGURE 8. Comparison of the pressure distribution and geometry for RAE2822 and optimized airfoils.

both algorithms, the number of nests is set to 20, and the p_a is set to 0.25. FIGURE 7 shows the averaged convergence histories of the drag coefficient by the original CICS and CS. It can be seen that CICS converges faster than CS. Besides, the standard deviation of the drag coefficient obtained by CICS is much smaller than by the CS, which indicates that CICS is more robust.

TABLE 19 shows the average optimal performance obtained by CICS and CS. We can see that CICS gets a smaller drag coefficient than CS, while all of the constraints are satisfied. FIGURE. 8 compares the pressure distribution (Cp) and the shape of RAE2822 and one of the optimized airfoils. It can be seen that, while there is a sharp shock on the upper bound of the baseline airfoil, the shock is removed for the optimized airfoil. As a result, the drag coefficient is dramatically decreased.

The aerodynamic design optimization case verifies that the constraint-handling improved cuckoo search algorithm is feasible to strongly constrained aerodynamic shape optimization problems, and it outperforms the original CS. It converges not only faster but also more accurate. Besides, it is more robust than the original CS (get a smaller standard deviation).

VI. DISCCUSION ON RESULTS

A. DISCUSSION

In this research, three work has been done. First, the performance of the presented CH method for meta-heuristic optimization algorithms is studied. Second, the performance

TABLE 19. Comparison of the optimization results by CICS and CS for airfoil design problem.

Airfoil	*Cd*	*Cl*	*Cm*	*Area*
RAE2822	0.022914	0.8240	-0.092721	0.077842
CS	0.014338 (-37.4%)	0.8240	-0.090875	0.078203
CICS	0.014246 (-37.8%)	0.8240	-0.092673	0.077877

of the constrained handling improved cuckoo search algorithm is considered. Last, the presented CICS algorithm is applied to a strongly constrained aerodynamic shape design optimization problem. Ten analytical benchmark problems and four structural design problems are employed to verify new algorithms. Besides, the Wilcoxon sign rank tests are introduced to compare the statistical results.

There are mainly two modifications of the proposed CH method, including the scaling of each constraint function and the dominance mechanism. TABLE 3 verifies the success of each change and the combination of them. Each of the two modifications can improve the performance compared to the original CH method, but they are not as good as the combination of them.

Five types of traditional CH methods, including the most frequently used penalty methods, and a famous separating method which is firstly proposed for genetic algorithm, are introduced for comparison to verify the new CH method. Different from most previous studies, the total number of function evaluations here is much smaller. The reason is that, for many real engineering design problems, the number of function evaluations is somewhat limited. For example, in the field of aerodynamic shape design optimization when the time-consuming computational fluid dynamics are involved. Besides, except for the optimal objective value, the feasibility ratio and the CPU time are also taken as performance metrics. From FIGURE 3 to FIGURE 4 and TABLE 4 to TABLE 6, the present CH method shows the best performance for most cases: it gets a 100% feasibility ratio for each problem. It obtains the smallest objective value for most problems. Deb's method performs second best. It gets a 100% feasibility ratio for most problems and usually the second smallest objective function value. For the penalty methods, they frequently don't find feasible solutions if we repeat the optimization process due to the algorithms' randomness and a small number of total function evaluations. Comparing the static penalty method with the default penalty factor ($PF = 10^{10}$) to the static penalty with the best PF (Each problem corresponds to a different value) further verifies that the PF strongly affects the optimization result. If the best value of PF is introduced, the results will be significantly improved, and it performs only second to the present method and Deb's method. For the CPU time, the difference between algorithms is tiny, which can be ignored in engineering practice. TABLE 4 to TABLE 6 also reveals that the size of the CS population does not affect the superiority of the proposed CH method. Still, it slightly alters the final result for a specified CH method. Additionally, the G02 problem results indicate that, if the constraint is weak for an optimization problem, almost any type of CH method is satisfactory. However, if the constraint is rather strong, a better CH method has to be chosen. The present CH method is also incorporated to improved PSO and GA, and gives a better solution than other CH methods. However, the advantage is not as remarkable as when integrating into the CS.

The presented CICS algorithm is then compared with nine types of other meta-heuristic algorithms, including improved GA, improved PSO, modified PSO, weighted DE, and different recent algorithms. From TABLE 18, we can conclude that the CICS performs best with a relatively small number of total function evaluations. There are mainly two reasons. First, the CS algorithm uses a combination of global exploration and local exploitation controlled by switching probability. Based on the switching strategy, local search becomes very intensive, occurring for only one-fourth of the time, and global search is extensive, taking place for around three-fourth of the time. Besides, the cuckoo search follows Lévy flight behavior, which is far better than a simple random walk. Lévy flights have infinite variance and mean, which helps CS explore the search space better. This advantage makes CS more efficient in achieving global convergences [40]. Second, by incorporating the new CH method, the CICS algorithm can eliminate the adverse effects of order of magnitude difference between different constraint functions, making it easier to find solutions that meet all constraints synchronously.

Finally, the CICS algorithm is applied to a transonic airfoil shape optimization. In this optimization problem, a function evaluation costs about one to two minutes on a personal computer; hence the total number of function evaluations should not be extensive. Consequently, we only compared it to the original CS while not the rest meta-heuristic algorithms. We can conclude that the CICS performs remarkably better than the original CS. The result also indicates that the proposed CICS is feasible for aerodynamic shape optimization problems, and the advantage of improvement can also benefit the aerodynamic shape design.

B. LIMITATIONS AND IMPLICATIONS

In general, compared with other algorithms introduced in this study, the CH method and CICS algorithm presented in this paper show better performance. However, these algorithms still have some limitations, and more work needs to do.

First, although the numerical experiments prove the superiority of the algorithm in this paper, there is still a lack of strict theoretical proof for the advantage of the algorithm.

Second, although the penalty methods and Deb's method are the most frequently used CH methods for meta-heuristic algorithms, there may still more modern methods that haven't been introduced for comparison. As a result, the advantages of the algorithm proposed in this paper are only relative to

the methods compared in this paper. For other CH methods, more work should be done. Similarly, the advantages of the CICS algorithm proposed in this paper are only relative to the algorithms compared in this research. As mentioned in the introduction, many meta-heuristic algorithms have been presented in recent years, but only a small part of them are used for comparison in this study. Hence, more comparison should be made to verify its performance further.

Third, unlike most previous studies in which the total number of function evaluations is huge (200000∼300000), the allowable number of function evaluations in this research is much smaller. It still needs more work to further study its performance by setting the permissible function evaluation as the same in other papers.

Last, the convergence rate of CICS should be further improved for three-dimensional aerodynamic shape optimization. The reason is that it usually takes several hours to calculate a single aerodynamic function. The total allowable number of function evaluations will be even less than in present research.

VII. CONCLUSION AND FUTURE WORKS

This research first presents an effective constraint-handling method for swarm-based meta-heuristic optimization algorithms, which utilizes the scaling of different constraint functions and a dominant mechanism for comparison. It has no user-defined parameter. A constraint-handling improved cuckoo search algorithm is then presented by applying the new CH method to the original unconstrained cuckoo search algorithm.

With the CS as an optimizer, the presented CH method is compared with five types of existing traditional methods, by employing ten benchmark problems and four structural design optimization problems. The results indicate that the new method significantly outperforms the conventional constraint-handling methods: it converges faster and more accurate and gets a higher percentage of feasible solutions. Besides, the Wilcoxon signed-rank test also supports the significant advantages of the new method over other ones. The newly proposed constraint-handling method is then applied to two standard meta-heuristic algorithms GA and PSO, respectively. Test cases also verify that the new constraint-handling method outperforms others no matter which optimizer is chosen.

The proposed constraint-handling improved cuckoo search algorithm is further studied by comparing it with nine other meta-heuristic optimization algorithms. The results indicate that the CICS outperforms the different optimization algorithms for most test problems.

Finally, the constraint-handling improved cuckoo search algorithm is then successfully applied to a strongly constrained drag minimization problem of a transonic airfoil. The results show that, compared to the original cuckoo search with the penalty method, the new method converges faster and gets a smaller drag coefficient.

In future works, the present CH method can be incorporated into the latest improved cuckoo search algorithms or other types of meta-heuristic optimization algorithms to improve the performance further. Besides, the current CICS algorithm can be extended to solve multi-objective problems; that is, some multi-objective constraint handling improved cuckoo search can be presented. Finally, the newly introduced CICS algorithm can be applied to other aerodynamic shape optimization problems or optimization problems in other fields.

REFERENCES

[1] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Ann Arbor, MI, USA: Univ. Michigan Press 1975.
[2] S. O. Orero and M. R. Irving, "A genetic algorithm modelling framework and solution technique for short term optimal hydrothermal scheduling," *IEEE Trans. Power Syst.*, vol. 13, no. 2, pp. 501–518, May 1988.
[3] S. He, E. Prempain, and Q. H. Wu, "An improved particle swarm optimization for mechanical design optimization problems," *J. Eng. Optim.*, vol. 36, no. 5, pp. 585–605, 2004.
[4] M. Dorigo and T. Stützle, "Ant colony optimization: Overview and recent advances," in *Handbook of Metaheuristics* (International Series in Operations Research & Management Science), vol. 146, M. Gendreau and J. Y. Potvin, Eds. Boston, MA, USA: Springer, 2010. [Online]. Available: https://doi.org/10.1007/978-1-4419-1665-5_8
[5] J. Liu, W.-P. Song, Z. H. Han, and Y. Zhang, "Efficient aerodynamic shape optimization of transonic wings using a parallel infilling strategy and surrogate models," *Struct. Multidisciplinary Optim.*, vol. 55, pp. 925–943, Aug. 2017.
[6] J. Liu, Z. H. Han, and W. P. Song, "Efficient kriging-based aerodynamic design of transonic airfoils: Some key issues," in *Proc. 50th AIAA Aerosp. Sci. Meeting Including New Horizons Forum Aerosp. Expo.*, Nashville, TN, USA, 2012, p. 967.
[7] Z. H. Han, C. Z. Xu, L. Zhang, Y. Zhang, K. S. Zhang, and W. P. Song, "Efficient aerodynamic shape optimization using variable-fidelity surrogate models and multilevel computational grids," *Chin. J. Aeronaut.*, vol. 33, no. 1, pp. 31–47, Jan. 2020, doi: 10.1016/j.cja.2019.05.001.
[8] D. Karaboga, "An idea based on honey bee swarm for numerical optimization," Engineering Faculty, Computer Engineering Department, Erciyes Univ., Kayseri, Turkey, Tech. Rep. TR06, 2005.
[9] R. Salgotra, U. Singh, S. Saha, and A. K. Nagar, "Improved flower pollination algorithm for linear antenna design problems," in *Soft Computing for Problem Solving* (Advances in Intelligent Systems and Computing), vol. 1048, K. Das, J. Bansal, K. Deep, A. Nagar, P. Pathipooranam, and R. Naidu, Eds. Singapore: Springer, 2020.
[10] A. Babalik, A. C. Cinar, and M. S. Kiran, "A modification of tree-seed algorithm using Deb's rules for constrained optimization," *Appl. Soft Comput.*, vol. 63, pp. 289–305, Feb. 2018.
[11] F. A. Hashim, E. H. Houssein, M. S. Mabrouk, W. Al-Atabany, and S. Mirjalili, "Henry gas solubility optimization: A novel physics-based algorithm," *Future Gener. Comput. Syst.*, vol. 101, pp. 646–667, Dec. 2019.
[12] R. Salgotra and U. Singh, "The naked mole-rat algorithm," *Neural Comput. Appl.*, vol. 31, no. 12, pp. 8837–8857, Dec. 2019, doi: 10.1007/s00521-019-04464-7.
[13] S. Li, H. Chen, M. Wang, A. A. Heidari, and S. Mirjalili, "Slime mould algorithm: A new method for stochastic optimization," *Future Gener. Comput. Syst.*, vol. 111, pp. 300–323, Oct. 2020, doi: 10.1016/j.future.2020.03.055.
[14] A. Faramarzi, M. Heidarinejad, B. Stephens, and S. Mirjalili, "Equilibrium optimizer: A novel optimization algorithm," *Knowl.-Based Syst.*, vol. 191, Mar. 2020, Art. no. 105190.
[15] A. A. Heidari, S. Mirjalili, H. Faris, I. Aljarah, M. Mafarja, and H. Chen, "Harris hawks optimization: Algorithm and applications," *Future Gener. Comput. Syst.*, vol. 97, pp. 849–872, Aug. 2019.
[16] S. Mirjalili, "Moth-flame optimization algorithm: A novel nature-inspired heuristic paradigm," *Knowl.-Based Syst.*, vol. 89, pp. 228–249, Nov. 2015.
[17] S. Mirjalili, A. H. Gandomi, S. Z. Mirjalili, S. Saremi, H. Faris, and S. M. Mirjalili, "Salp Salp Swarm Algorithm: A bio-inspired optimizer for engineering design problems," *Adv. Eng. Softw.*, vol. 114, pp. 163–191, Dec. 2017.

[18] R. Salgotra, U. Singh, and S. Saha, "On some improved versions of whale optimization algorithm," *Arabian J. for Sci. Eng.*, vol. 44, no. 11, pp. 9653–9691, Nov. 2019, doi: 10.1007/s13369-019-04016-0.

[19] X.-S. Yang and S. Deb, "Cuckoo search via Lévy flights," in *Proc. World Congr. Nature Biol. Inspired Comput. (NaBIC)*, Dec. 2009, pp. 210–214.

[20] X. S. Yang and S. Deb, "Engineering optimization by cuckoo search," *Int. J. Math. Model. Numer. Optim.*, vol. 1, no. 4, pp. 330–343, Dec. 2010.

[21] X.-S. Yang and S. Deb, "Cuckoo search: Recent advances and applications," *Neural Comput. Appl.*, vol. 24, no. 1, pp. 169–174, Jan. 2014.

[22] X. S. Yang, *Nature-Inspired Metaheuristic Algorithms* 2nd. Beckington, U.K.: Luniver Press, 2010.

[23] A. H. Gandomi, X.-S. Yang, and A. H. Alavi, "Cuckoo search algorithm: A Metaheuristic approach to solve structural optimization problems," *Eng. with Comput.*, vol. 29, no. 1, pp. 17–35, Jan. 2013.

[24] A. Kaveh, T. Bakhshpoori, and M. Ashoory, "An efficient optimization procedure based on cuckoo search algorithm for practical design of steel structures," *Int. J. Optim. Civil Eng.*, vol. 2, no. 1, pp. 1–14, 2012.

[25] D. S. Naumann, B. Evans, S. Walton, and O. Hassan, "A novel implementation of computational aerodynamic shape optimisation using modified cuckoo search," *Appl. Math. Model.*, vol. 40, nos. 7–8, pp. 4543–4559, 2016.

[26] C. D. Tran, T. T. Dao, V. S. Vo, and T. T. Nguyen, "Economic load dispatch with multiple fuel options and valve point effect using cuckoo search algorithm with different distributions," *Int. J. Hybrid Inf. Technol.*, vol. 8, no. 1, pp. 305–316, Jan. 2015.

[27] D. N. Vo, P. Schegner, and W. Ongsakul, "Cuckoo search algorithm for non-convex economic dispatch," *IET Gener., Transmiss. Distrib.*, vol. 7, no. 6, pp. 645–654, 2013.

[28] X. Liu and H. Fu, "PSO-based support vector machine with cuckoo search technique for clinical disease diagnoses," *Sci. World J.*, vol. 2014, pp. 1–7, 2014.

[29] S. M. Abd Elazim and E. S. Ali, "Optimal power system stabilizers design via cuckoo search algorithm," *Int. J. Electr. Power Energy Syst.*, vol. 75, pp. 99–107, Feb. 2016.

[30] C. Cobos, H. Muñoz-Collazos, R. Urbano-Muñoz, M. Mendoza, E. León, and E. Herrera-Viedma, "Clustering of Web search results based on the cuckoo search algorithm and balanced Bayesian information criterion," *Inf. Sci.*, vol. 281, pp. 248–264, Oct. 2014.

[31] S. Pare, A. Kumar, V. Bajaj, and G. K. Singh, "A multilevel color image segmentation technique based on cuckoo search algorithm and energy curve," *Appl. Soft Comput.*, vol. 47, pp. 76–102, Oct. 2016.

[32] A. Gherboudj, A. Layeb, and S. Chikhi, "Solving 0-1 knapsack problems by a discrete binary version of cuckoo search algorithm," *Int. J. Bio-Inspired Comput.*, vol. 4, no. 4, pp. 229–236, 2012.

[33] X. Ouyang, Y. Zhou, Q. Luo, and H. Chen, "A novel discrete cuckoo search algorithm for spherical traveling salesman problem," *Appl. Math. Inf. Sci.*, vol. 7, no. 2, pp. 777–784, Mar. 2013.

[34] M. Tuba, M. Subotic, and N. Stanarevic, "Modified cuckoo search algorithm for unconstrained optimization problems," in *Proc. 5th Eur. Conf. Eur. Comput. Conf., World Sci. Eng. Academic Soc. (WSEAS)*, 2011, pp. 263–268.

[35] S. Walton, O. Hassan, K. Morgan, and M. R. Brown, "Modified cuckoo search: A new gradient free optimisation algorithm," *Chaos, Solitons Fractals*, vol. 44, no. 9, pp. 710–718, Sep. 2011.

[36] Y. Zhou, H. Zheng, and J. Wu, "An improved cuckoo search algorithm for solving planar graph coloring problem," *Appl. Math. Inf. Sci.*, vol. 7, no. 2, pp. 785–792, Mar. 2013.

[37] G. Kanagaraj, S. G. Ponnambalam, N. Jawahar, and N. J. Mukund, "An effective hybrid cuckoo search and genetic algorithm for constrained engineering design optimization," *Eng. Optim.*, vol. 46, no. 10, pp. 1331–1351, 2014.

[38] X.-S. Yang and S. Deb, "Multiobjective cuckoo search for design optimization," *Comput. Oper. Res.*, vol. 40, no. 6, pp. 1616–1624, Jun. 2013.

[39] X. S. Zhang, J. Liu, and S. B. Luo, "An improved multi-objective cuckoo search algorithm for airfoil aerodynamic optimization design," *Acta Aeronautica et Astronautica Sinica*, vol. 40, no. 6, 2019, Art. no. 122550.

[40] R. Salgotra, U. Singh, and S. Saha, "New cuckoo search algorithms with enhanced exploration and exploitation properties," *Expert Syst. Appl.*, vol. 95, pp. 384–420, Apr. 2018.

[41] R. Salgotra, U. Singh, and S. Saha, "Improved cuckoo search with better search capabilities for solving CEC2017 benchmark problems," in *Proc. IEEE Congr. Evol. Comput. (CEC)*, Jul. 2018, pp. 1–7, doi: 10.1109/CEC.2018.8477655.

[42] H. Chiroma, T. Herawan, I. Fister, I. Fister, S. Abdulkareem, L. Shuib, M. F. Hamza, Y. Saadi, and A. Abubakar, "Bio-inspired computation: Recent development on the modifications of the cuckoo search algorithm," *Appl. Soft Comput.*, vol. 61, pp. 149–173, Dec. 2017.

[43] M. Shehab, A. T. Khader, and M. A. Al-Betar, "A survey on applications and variants of the cuckoo search algorithm," *Appl. Soft Comput.*, vol. 61, pp. 1041–1059, Dec. 2017.

[44] C. A. Coello Coello, "Theoretical and numerical constraint-handling techniques used with evolutionary algorithms: A survey of the state of the art," *Comput. Methods Appl. Mech. Eng.*, vol. 191, nos. 11–12, pp. 1245–1287, Jan. 2002.

[45] A. Homaifar, C. X. Qi, and S. H. Lai, "Constrained optimization via genetic algorithms," *Simulation*, vol. 62, no. 4, pp. 242–253, Apr. 1994.

[46] S. Puzzi and A. Carpinteri, "A double-multiplicative dynamic penalty approach for constrained evolutionary optimization," *Struct. Multidisciplinary Optim.*, vol. 35, no. 5, pp. 431–445, May 2008.

[47] Y. Wang, Z. X. Cai, Y. R. Zhou, and Z. Fan, "Constrained optimization based on hybrid evolutionary algorithm and adaptive constraint-handling technique," *Struct. Multidisciplinary Optim.*, vol. 37, no. 4, pp. 395–413, 2009.

[48] K. Deb, "An efficient constraint handling method for genetic algorithms," *Comput. Methods Appl. Mech. Eng.*, vol. 186, pp. 311–338, Jun. 2000.

[49] T. P. Runarsson and X. Yao, "Stochastic ranking for constrained evolutionary optimization," *IEEE Trans. Evol. Comput.*, vol. 4, no. 3, pp. 284–294, 2000.

[50] T. Takahama and S. Sakai, "Constrained optimization by applying the α constrained method to the nonlinear simplex method with mutations," *IEEE Trans. Evol. Comput.*, vol. 9, no. 5, pp. 437–451, Oct. 2005.

[51] L. Wang and L.-P. Li, "An effective differential evolution with level comparison for constrained engineering design," *Struct. Multidisciplinary Optim.*, vol. 41, pp. 947–963, Dec. 2010.

[52] Z. Cai and Y. Wang, "A multiobjective optimization-based evolutionary algorithm for constrained optimization," *IEEE Trans. Evol. Comput.*, vol. 10, no. 6, pp. 658–675, Dec. 2006.

[53] G. Venter and R. Haftka, "Constrained particle swarm optimization using a bi-objective formulation," *Struct. Multidisciplinary Optim.*, vol. 40, p. 65, Apr. 2010.

[54] T. T. Nguyen, D. N. Vo, and A. V. Truong, "Cuckoo search algorithm for short-term hydrothermal scheduling," *Appl. Energy*, vol. 132, pp. 276–287, Nov. 2014.

[55] R. Mallipeddi and P. N. Suganthan, "Ensemble of constraint handling techniques," *IEEE Trans. Evol. Comput.*, vol. 14, no. 4, pp. 561–579, Aug. 2010.

[56] Y. Shi, "Particle swarm optimization: Developments, applications and resources," in *Proc. Congr. Evol. Comput.*, vol. 1, May 2001, pp. 81–86.

[57] J. Derrac, S. García, D. Molina, and F. Herrera, "A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms," *Swarm Evol. Comput.*, vol. 1, no. 1, pp. 3–18, Mar. 2011.

[58] X. S. Yang, "Firefly algorithms for multimodal optimization," in *Proc. Int. Symp. Stochastic Algorithms*. Berlin, Germany: Springer, 2009, pp. 169–178.

[59] S. Mirjalili, S. M. Mirjalili, and A. Lewis, "Grey wolf optimizer," *Adv. Eng. Softw.*, vol. 69, pp. 46–61, Mar. 2014.

[60] D. Karaboga and B. Akay, "A modified artificial bee colony (ABC) algorithm for constrained optimization problems," *Appl. Soft Comput.*, vol. 11, no. 3, pp. 3021–3031, 2011.

[61] P. Civicioglu, E. Besdok, M. A. Gunen, and U. H. Atasever, "Weighted differential evolution algorithm for numerical function optimization: A comparative study with cuckoo search, artificial bee colony, adaptive differential evolution, and backtracking search optimization algorithms," *Neural Comput. Appl.*, vol. 32, no. 8, pp. 3923–3937, Apr. 2020.

[62] S. Mirjalili, A. Lewis, and A. S. Sadiq, "Autonomous particles groups for particle swarm optimization," *Arabian J. for Sci. Eng.*, vol. 39, no. 6, pp. 4683–4697, Jun. 2014.

[63] J. Ahn, K. Yee, and D.-H. Lee, "Two-point design optimization of transonic airfoil using response surface methodology," in *Proc. 37th Aerosp. Sci. Meeting Exhibit*, Jan. 1999, p. 403.

[64] B. M. Kulfan, "Universal parametric geometry representation method," *J. Aircr.*, vol. 45, no. 1, pp. 142–158, Jan. 2008.



JUN LIU received the B.S. and Ph.D. degrees from the School of Aeronautics, Northwestern Polytechnical University, Xi'an, China, in 2009 and 2015, respectively. He is currently a Lecturer with the School of Aeronautics and Astronautics, Central South University. His current research interests include hypersonic aerodynamics, evolutionary optimization algorithms, and aerodynamic shape optimization and their practical applications.



DOU WANG received the M.S. and Ph.D. degrees from the School of Transportation Engineering, Beihang University, Beijing, China, in 2013 and 2018, respectively. He is currently a Lecturer with the School of Aeronautics and Astronautics, Central South University. His current research interests include hypersonic aerodynamics and bionic unsteady aerodynamic and their practical applications.



SHIBIN LUO received the B.S., M.S., and Ph.D. degrees from the School of Aerospace Engineering, National University of Defense Science and Technology, Changsha, China, in 1998, 2000, and 2004, respectively. He is currently a Professor and the Dean of the School of Aeronautics and Astronautics, Central South University. His current research interests include hypersonic aerodynamics and hypersonic aircraft designing.